

Medi-Reach

(Emergency Donor Network)

A Project Report

Submitted in partial fulfilment of the
Requirements for the award of degree of
BACHELOR OF VOCATION (SOFTWARE DEVELOPMENT)

BY

Name: Sahil Modi

UID: 23BSD021

Under the esteemed guidance of Prof

Mr. Wilson Rao

Co-ordinator and

Ms. Taiba Habshi

Assistant Professor

DEPARTMENT OF SOFTWARE DEVELOPMENT

JAI HIND COLLEGE



(Empowered *Autonomous*)

MUMBAI, 400020

MAHARASHTRA

2025-26

JAI HIND COLLEGE

(Empowered Autonomous)

MUMBAI, 400020 MAHARASHTRA

DEPARTMENT OF SOFTWARE DEVELOPMENT



CERTIFICATE

This is to certify that the project entitled, “Medi-Reach” is bonafied work of Sahil Modi bearing UID (23BSD021) submitted in partial fulfillment of the requirements for the award of degree of BACHELOR OF VOCATION IN SOFTWARE DEVELOPMENT from Jai Hind College Autonomous (University of Mumbai).

Internal Guide

Coordinator

Date

External Examiner

College Seal

DECLARATION

I hereby declare that the project entitled, “Medi-Reach” done at Jai Hind College, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfilment of the requirements for the award of degree of BACHELOR OF VOCATION (SOFTWARE DEVELOPMENT) to be submitted as final semester project as part of our curriculum.

Name and Signature of the Student

ACKNOWLEDGEMENT

I would like to express my thanks to the people who have helped me most throughout my project. I am grateful to my guide Ms. Taiba Habshi for nonstop support for the project. I can't say thank you enough for her tremendous support and help.

I owe my deep gratitude to our HOD of SOFTWARE DEVELOPMENT Department Mr. Wilson Rao who took keen interest in our project work and guided us all along till the completion of our project work by providing all the necessary information for developing a good system.

At last, but not least I want to thank all my friends who helped/treasured me out in completing the project, where they all exchanged their own interesting ideas and thoughts and made this possible to complete my project with all accurate information. I wish to thank my parents for their personal support or attention who inspired/encouraged me to go my own way.

ABSTRACT

Medi-Reach is a comprehensive emergency medical resource and donor network system designed to streamline the interaction between individuals in urgent medical need, verified donors, NGOs, hospitals, and administrators on a unified digital platform. The application offers essential features such as secure user registration, emergency request creation and tracking, donor availability management, NGO/Admin verification workflows, AI-based donor matching, and real-time notification services. A distinctive aspect of Medi-Reach is its role-based access system, ensuring that receivers, donors, NGOs, and administrators each have customized dashboards with relevant functionalities and controlled permissions. The inclusion of request and user verification mechanisms enhances authenticity and prevents fraudulent or misleading emergency posts, while the integration of geolocation services and AI models ensures faster and more accurate donor matching. Built using modern cloud technologies and real-time databases, Medi-Reach ensures scalability, reliability, and accessibility across devices. Designed with a user-friendly interface and strong data security measures, Medi-Reach aims to reduce response time during medical emergencies, improve coordination among stakeholders, and promote transparency and trust within the emergency healthcare support ecosystem.

INDEX

Sr No			PARTICULARS	PAGE NO.	DATE
1			Introduction	1-6	4-12-2025
	1.1		Background	1	
	1.2		Objectives	2	
	1.3		Purpose, Scope, and Applicability	3	
		1.3.1	Purpose	3	
		1.3.2	Scope	3	
		1.3.3	Applicability	3	
	1.4		Achievements	4	
	1.5		Organization of Report	5	
2			Survey Of Technologies	7 - 10	13-12-2025
3			Requirement And Analysis	11-31	28-12-2025
	3.1		Problem Definition	11	
	3.2		Requirement Specification	12	
	3.3		Planning and Scheduling	14	
	3.4		Software and Hardware Requirements	15	
	3.5		Preliminary Product Description	16	

	3.6		Conceptual Models	17	11-12-2025
		3.6.1	Event Table	17	
		3.6.2	Use Case Diagram	19	
		3.6.3	Entity Relationship Diagram	20	
		3.6.4	Class Diagram	21	21-12-2025
		3.6.5	Object Diagram	22	
		3.6.6	Activity Diagram	23	
		3.6.7	Sequence Diagram	24	26-12-2025
		3.6.8	State Flow Diagram	25	
		3.6.9	Context Diagram	26	
		3.6.10	Data Flow Diagram	27	31-08-2024
		3.6.11	Component Diagram	29	
		3.6.12	Package Diagram	30	
		3.6.13	Deployment Diagram	31	
4			System Coding	32-54	14-01-2026
	4.1		Code	32-45	
	4.2		Data Dictionary	46-47	
	4.3		Program Description	48-50	
	4.4		Naming Conventions	51-53	
	4.5		Validations	54	

5			Program Listing	55-65	21-01-2026
	5.1		Cost Estimation	55-56	
	5.2		Schema Design	57-60	
	5.3		User Manual with Screenshots	61-65	
	5.4		Test Cases Design	66	
6			Conclusion	67-68	28-01-2026
	6.1		Conclusion	67	
	6.2		Limitations of the System	67	
	6.3		Future Scope of the Project	68	
7			Bibliography	69	28-01-2026

INTRODUCTION

1.1 - BACKGROUND

Medi-Reach is a digital emergency-response platform designed to connect people in medical need with verified donors, NGOs, hospitals, and emergency resource providers. It addresses a critical gap in the healthcare system where emergency requests for blood, oxygen, rare medicines, or supplies are scattered, unreliable, and often unverified. In many cases, individuals rely on social media posts that may be fake, outdated, or non-actionable, resulting in delays and life-threatening consequences. Medi-Reach offers a unified, authenticated, and AI-assisted system that verifies requests, identifies legitimate donors, and matches them in real-time using geolocation and AI models. The platform enhances transparency, reduces misinformation, and accelerates emergency response efforts. It aligns with global digital health and humanitarian initiatives that focus on rapid, reliable, and secure resource delivery during crises, natural disasters, pandemics, and medical emergencies.

1.2 OBJECTIVES -

- Create a centralized emergency assistance platform for real-time donor–receiver connection.
- Improve coordination between donors, receivers, NGOs, and hospitals through verified data sharing.
- Provide a secure, role-based authentication system for all user types.
- Enable users to raise emergency requests that are quickly verified and prioritized.
- Use AI/ML models to predict urgency levels and suggest the nearest available donors.
- Streamline communication through notifications and live request tracking.
- Enhance transparency to eliminate fraudulent donation posts and misinformation.
- Build a scalable platform that supports national and international humanitarian networks.

1.3- PURPOSE, SCOPE AND APPLICABILITY

1.3.1- PURPOSE

The purpose of Medi-Reach is to provide a lifesaving, secure, and digital emergency resource system that connects verified donors and organizations with individuals in urgent need. It aims to reduce response time, eliminate fake donation posts, ensure authenticity of requests and donors, and support faster medical assistance through an efficient donor-matching algorithm. By digitizing and validating emergency communication, Medi-Reach fosters trust, reduces human error, and accelerates access to critical medical resources.

1.3.2- SCOPE

Medi-Reach is designed as a comprehensive emergency assistance network that integrates real-time donor matching, request verification, AI-based prioritization, and map-based locating. Its scope includes:

- Secure authentication for donors, receivers, NGOs, and admins
- Real-time donor matching based on geolocation.
- Request creation, verification, and tracking.
- Resource availability updates by donors and NGOs
- Push notifications for urgent alerts and donor matches.
- AI-driven urgency prediction and donor recommendation
- A global-ready directory for donors, hospitals, blood banks, and relief organizations

The platform supports integration with national-level health infrastructures and humanitarian disaster relief programs. By prioritizing authenticity, scalability, and accessibility, Medi-Reach improves emergency-response operations for both individuals and organizations.

1.3.2- APPLICABILITY

For Receivers (Patients / Families):

Medi-Reach empowers individuals during critical situations by allowing them to post verified emergency requests such as blood, plasma, oxygen, or life-saving medicines. Users can track their request status in real-time, view matched donors, and receive instant notifications when help is nearby. The platform eliminates dependency on unreliable social media posts and ensures that only authenticated donors respond.

For Donors:

Donors can register their availability, update their status, and receive real-time alerts about nearby verified emergencies. The AI model ensures donors are matched based on proximity, availability, and relevance. Medi-Reach allows donors to contribute safely, transparently, and efficiently without being overwhelmed by fake or unverified requests.

For NGOs and Hospitals:

NGOs and hospitals can verify emergency requests, update resource availability, and coordinate medically critical cases. They have access to region-based dashboards, donor lists, and tracking tools that support faster decision-making. This improves operational workflows, strengthens emergency preparedness, and enhances public trust.

For Admins:

Admins ensure platform integrity by authorizing NGOs, validating requests, removing fraudulent users, and overseeing the system's operations. They ensure transparency, compliance, and efficient functioning of all modules.

1.4- ACHIEVEMENTS

Medi-Reach offers a unique emergency medical assistance experience by combining personalized user features with a structured administrative and verification system for enhanced coordination and reliability. Unlike informal social media-based request systems, Medi-Reach provides verified emergency request handling, real-time donor matching, and intelligent urgency prioritization through AI integration. The platform enables receivers to track the status of their requests, allows donors to manage availability and respond efficiently, and empowers NGOs and administrators to validate and monitor cases transparently. By bridging the gap between individuals in need and verified medical resources, Medi-Reach ensures faster response times, improved trust, and organized emergency support management within a secure and scalable digital ecosystem.

1.5- ORGANIZATION OF REPORT

1.5.1- REQUIREMENT AND ANALYSIS:

- 1] The frontend of MEDI-REACH is built using React.js, ensuring a fast, component-driven, and responsive user interface. For styling, it uses Tailwind CSS or Material UI, both of which support clean, modern, and accessible designs suitable for healthcare environments.
- 2] The backend is powered by Node.js, with APIs handling user authentication, data processing, and integration logic. Firebase is used for:
- 3] Authentication: Secure login/signup with role-based access.
- 4] Firestore: Real-time database for storing patient records, lab results, and claims.
- 5] Deployment and hosting are handled via Vercel, which supports serverless backend functions and continuous deployment with GitHub integration.
- 6] To develop and maintain the application, the following are the recommended system specifications: Intel i5 or equivalent (minimum), 8 GB or more Windows 10 / macOS / Linux, Chrome, Firefox, or any modern browser, Stable internet connection for Firebase and Vercel operations.
- 7] Users accessing the system only require a stable internet connection and a browser - no installation is needed.

1.5.2 - SYSTEM DESIGN

- The system design is included in all conceptual schema diagrams which explain the inter-relation between the modules and their dependencies.
- The features, working and flow of the entire system are described in this chapter.

1.5.3- IMPLEMENTATION AND TESTING

- Implementation process is described in this chapter.
- Describes which all testing approaches are going to take place in this project.

1.5.4 - RESULTS AND DISCUSSION

The Medi-Reach platform successfully addresses critical challenges in emergency medical resource coordination by providing a centralized, real-time, and verified digital system accessible to receivers, donors, NGOs, and administrators. Its responsive and role-based interface enables users to interact with personalized dashboards, create and monitor emergency requests, manage donor availability, and track request status with clarity and transparency. The integration of Firebase ensures secure authentication and real-time data synchronization, allowing instant updates on request verification, donor matching, and status changes. The use of React and Node.js provides dynamic and scalable front-end-backend architecture capable of handling multiple concurrent users efficiently. The AI-based donor matching mechanism enhances response speed by prioritizing urgent cases and identifying nearby available donors using geolocation logic. The verification module significantly reduces misinformation and fraudulent emergency posts, improving trust and reliability within the system. Overall, Medi-Reach achieves its objective of improving emergency response efficiency, transparency, and coordination within the healthcare support ecosystem.

1.5.5 CONCLUSION

Medi-Reach successfully fulfills its intended purpose of digitizing and organizing emergency medical assistance by streamlining the interaction between individuals in need, donors, NGOs, and administrators. The platform provides a user-friendly and efficient interface supported by a robust backend infrastructure, ensuring secure, reliability, and real-time coordination during medical emergencies. By integrating authentication systems, verification workflows, AI-based donor matching, and notification services, Medi-Reach enhances decision-making, reduces response time, and minimizes misinformation. Its scalable cloud-based architecture allows future expansion and integration with national health networks, disaster management systems, and government registries. With its structured design and scope for advanced AI and interoperability enhancements, Medi-Reach lays a strong foundation for improving emergency healthcare coordination and promoting digital transformation in humanitarian medical response systems.

2. SURVEY OF TECHNOLOGIES

2.1 Front-End Technologies

2.1.1 User Interface Frameworks

React.js

React.js is a JavaScript-based front-end library used to build dynamic and responsive user interfaces. It follows a component-based architecture where the UI is divided into reusable components such as login forms, dashboards, request cards, and notification panels. React uses a **Virtual DOM**, which updates only the changed components instead of reloading the entire page, making the application fast and efficient. This is especially useful in Medi-Reach where real-time updates are required for emergency requests, donor availability, and notification alerts.

- State management for tracking request status and user roles
- Conditional rendering for role-based dashboards (Donor, Receiver, NGO, Admin)
- Easy integration with Firebase services and REST APIs

ReactRouter

React Router is used for client-side routing and navigation. It enables smooth transitions between pages such as login, dashboard, request creation, and directory views without page reloads.

It also supports **protected routes**, ensuring that only authorized users can access specific modules.

2.1.2 Front-End Design Tools

Tailwind-CSS

Tailwind CSS is a utility-first CSS framework used to design responsive and accessible interfaces. Instead of writing custom CSS, predefined utility classes are used to style components. That includes Faster UI development, Consistent design across pages, Mobile-first responsiveness, Clean and readable layouts suitable for emergency-focused applications

Material UI (MUI)

Material UI is used optionally to implement ready-made components such as alerts, modals, dialogs, buttons, and form elements. These components follow Google's Material Design guidelines, improving usability and accessibility.

Leaflet.js / React-Leaflet

Leaflet is an open-source JavaScript library used for interactive maps. Medi-Reach uses Leaflet to display:

- Emergency request locations
- Donor locations
- Hospitals and NGOs

Markers, popups, and radius-based search visualization help users quickly locate nearby help. React-Leaflet allows seamless integration of Leaflet maps within React components.

2.2 Back-End Technologies

2.2.1 Programming Language

Node.js

Node.js is used as the backend runtime environment. It allows JavaScript to be executed on the server side. Node.js follows a **non-blocking, event-driven architecture**, making it ideal for real-time applications like Medi-Reach where multiple users interact simultaneously.

Key benefits:

- Handles multiple API requests efficiently
- Supports real-time notifications and updates
- Easily integrates with Firebase and AI microservices

Express.js

Express.js is a minimal backend framework built on top of Node.js. It is used to:

- Creating RESTful APIs
- Handle routing and middleware
- Validate requests
- Communicate with Firebase and AI services

Express simplifies backend development while maintaining flexibility and scalability.

2.2.2 Machine Learning Libraries

Scikit-learn

Scikit-learn is a Python-based machine learning library used for implementing classification and matching algorithms. It provides ready-to-use algorithms like Logistic Regression, KNN, Random Forest, and clustering models.

In Medi-Reach, Scikit-learn is used for:

- Urgency classification of emergency requests
- Donor ranking based on multiple parameters

NumPy

NumPy is used for numerical computations such as distance calculations, array manipulation, and mathematical operations required in ML models.

Pandas

Pandas is used for data manipulation and preprocessing. It helps clean, filter, and transform datasets before feeding them into ML models.

Geopy

Geopy is a Python library used to calculate geographical distances between donor and receiver locations using latitude and longitude values. It helps determine the nearest donors efficiently.

Flask

Flask is a lightweight Python web framework used to deploy ML models as a REST API. The Node.js backend communicates with Flask to get predictions and donor match results.

2.2.3 Database Systems

Firebase Firestore

Firestore is a cloud-based NoSQL database used to store, Firestore supports **real-time data synchronization**, allowing instant updates on request status and donor matches. It is scalable, secure, and serverless, making it suitable for global applications.

- User profiles
- Emergency requests
- Donor availability
- Verification logs
- Notification records

Firebase Storage

Used for storing verification documents, NGO approvals, and identity proofs securely.

2.3 AI/ML Technologies

2.3.1 Data Collection Tools

System-Generated Data

Medi-Reach collects structured data such as:

- Request type and urgency
- Timestamp of emergency
- Location coordinates
- Donor response times
- Fulfillment success rate

This data is continuously stored in Firestore and later used to train ML models.

Open-source Emergency Datasets (Future Scope)

Public datasets related to healthcare emergencies and population density can be integrated to improve prediction accuracy.

2.3.2 Data Pre-processing Methods

Data preprocessing ensures model accuracy and reliability.

- **Data Cleaning:**
Removal of duplicate, incomplete, or invalid entries.
- **Feature Engineering:**
Converting raw data into useful features such as:
 - Distance between donor and receiver

- Resource criticality score
- Time-based urgency factor
- **Normalization & Scaling:**
Ensures numerical values remain within comparable ranges.
- **Encoding:**
Converts categorical values (blood type, resource type) into numerical formats.
- **Handling Missing Data:**
Using default values or removing unusable records.

2.3.3 Algorithms / Models Used

1. K-Nearest Neighbors (KNN)

KNN is used to find the closest donors based on geographic distance.

Working:

- Calculating distance between receiver and all donors
- Select the nearest K donors
- Ranks them based on proximity and availability

2. Logistic Regression

Used to classify emergency urgency into categories such as Low, Medium, and Critical. It predicts urgency based on input features like resource type, location, and time sensitivity.

3. Random Forest (Optional Enhancement)

Random Forest combines multiple decision trees to improve prediction accuracy and reduce overfitting.

4. Clustering (K-Means)

Used to group donors by geographical region.

This improves performance by limiting donor search to relevant clusters.

5. Custom Weighted Scoring Algorithm

Final donor ranking is done using a scoring formula:

Final Score = Distance Weight + Availability Score + Reliability Factor

3. REQUIREMENT AND ANALYSIS

3.1- PROBLEM DEFINITION

In medical emergency situations, individuals often face extreme difficulty in finding verified donors, hospitals, NGOs, or emergency medical resources such as blood, oxygen, plasma, or essential medicines. Currently, emergency requests are largely circulated through social media platforms or informal networks, which lack authenticity, verification, and real-time coordination. This leads to misinformation, delayed responses, duplication of efforts, and, in many cases, loss of life. There is no centralized, secure, and real-time system that connects people in urgent medical need with verified donors and organizations based on location, availability, and urgency. The absence of proper verification mechanisms increases the risk of fake donation requests and donor fraud. Additionally, the lack of intelligent prioritization makes it difficult to address critical cases efficiently. Therefore, there is a need for a unified, transparent, and AI-enabled platform that can manage emergency medical requests, verify participants, and enable fast donor–receiver matching while ensuring data security and reliability.

3.1.1- EXISTING SYSTEM

The existing system for emergency medical assistance is largely unstructured and decentralized. People rely heavily on social media posts, messaging groups, or personal contacts to request urgent medical help. These platforms do not provide request verification, donor authentication, or real-time tracking. As a result, users often encounter fake or outdated information, unresponsive donors, and lack of coordination. Hospitals and NGOs manage emergency cases independently, without a common platform to share donor availability or urgent requirements. Donor databases, if available, are often static, outdated, and inaccessible to the public. There is no automated mechanism to prioritize emergencies based on severity or proximity. Manual coordination leads to delays, inefficiency, and increased burden on healthcare providers and volunteers. Overall, the existing system lacks reliability, transparency, scalability, and technological support.

3.1.2- PROPOSED SYSTEM

Medi-Reach is a web-based emergency medical resource and donor network designed to connect people in urgent need with verified donors, NGOs, and hospitals in real time. Medi-Reach provides role-based dashboards for donors, receivers, NGOs, and administrators, ensuring secure access and controlled data sharing. Users can create emergency requests specifying the required resource, location, and urgency. Requests are verified by NGOs or administrators before being made active. System integrates AI-based donor matching that identifies nearby available donors using geolocation data and prioritizes cases based on urgency. Real-time notifications are sent to matched donors, enabling faster responses. It ensures transparency through request tracking, verification logs, and status updates. The platform uses modern web technologies, real-time databases, and notification services to ensure reliability and scalability. The system is designed to support future integration with government health systems, disaster response platforms, and national donor registries.

3.2 – REQUIREMENT SPECIFICATION

FUNCTIONAL REQUIREMENTS

I. User Authentication

The system must provide secure login and registration for donors, receivers, NGOs, and administrators using role-based access control.

II. Emergency Request Management

Users must be able to create, update, and track emergency medical requests such as blood, oxygen, plasma, or medicines.

III. Verification System

NGOs and administrators must be able to verify requests and donors to ensure authenticity and prevent fraud.

IV. Donor Availability Management

Donors must be able to update their availability status and resource details in real time.

V. AI-Based Donor Matching

The system should intelligently match emergency requests with the nearest suitable donors based on location, urgency, and availability.

VI. Notification System

The system must send real-time notifications to donors and receivers regarding request updates, matches, and completion status.

VII. Request Tracking

Users must be able to track the status of requests (Pending, Verified, In Progress, Completed) in real time.

VIII. Responsive Design

The platform must function seamlessly on mobile phones, tablets, and desktop devices.

IX. Real-Time Data Synchronization

All updates must reflect instantly for authorized users using real-time database synchronization.

NON-FUNCTIONAL REQUIREMENTS

I. Performance:

The system must handle multiple concurrent users and emergency requests without performance degradation.

II. Scalability:

The platform should be scalable to support a growing number of users, donors, and organizations across regions.

III. Security:

User data and communications must be protected using encryption, secure authentication, and role-based access control.

IV. Usability:

The system should provide a simple, intuitive, and user-friendly interface suitable for emergency situations.

V. Reliability:

The application must ensure high availability with minimal downtime and robust error-handling mechanisms.

VI. Compatibility:

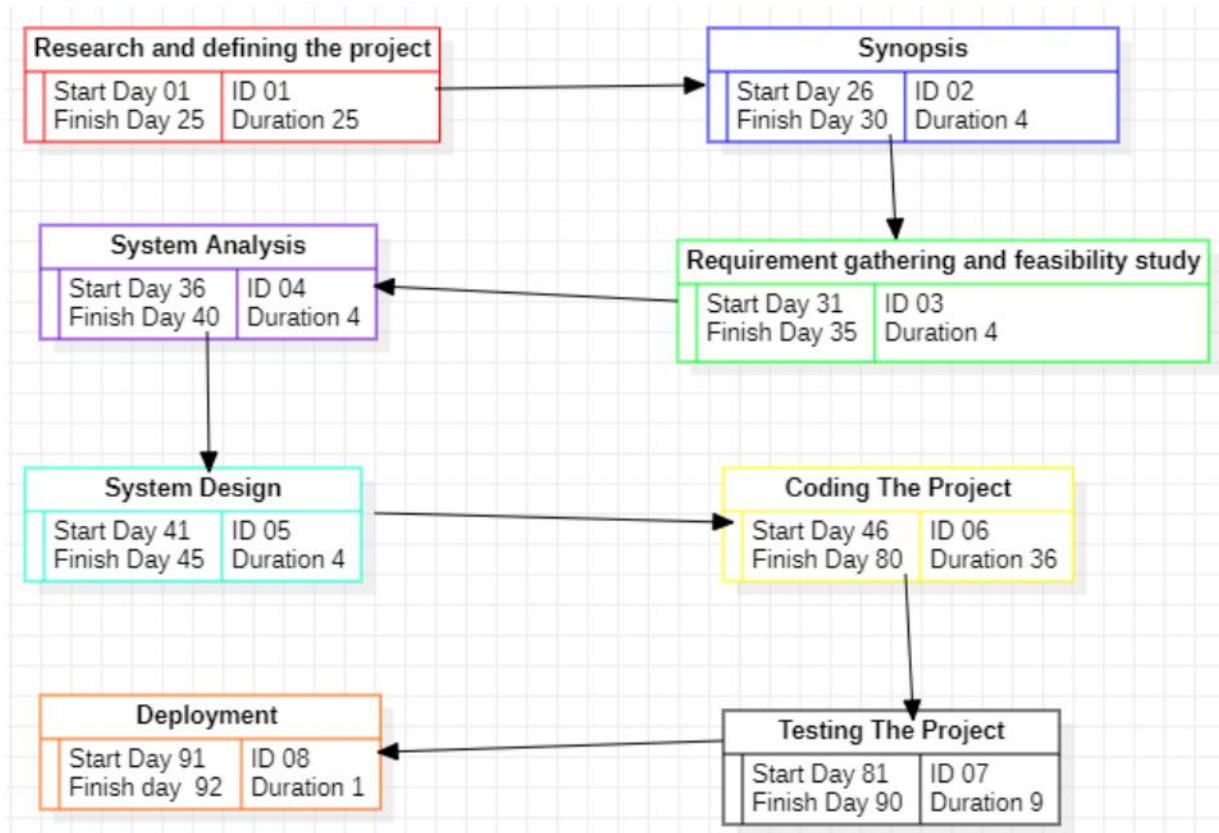
The system should be compatible with various devices, browsers, and operating systems.

VII. Maintainability:

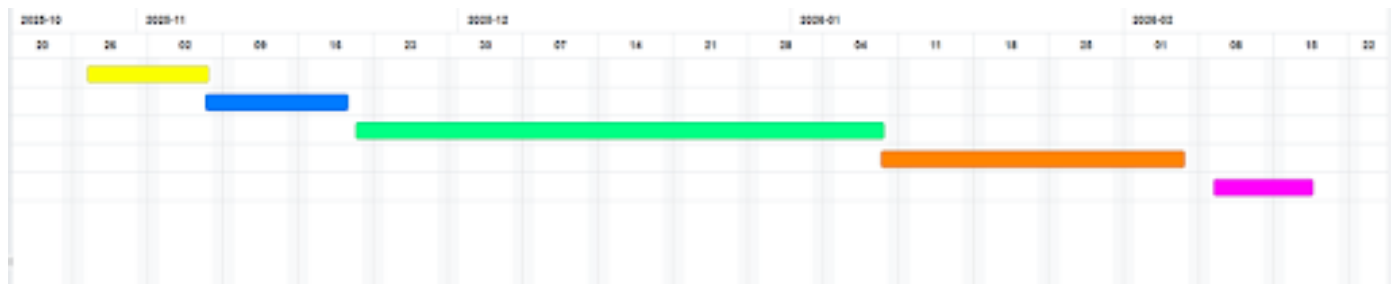
The codebase should be modular, well-documented, and easy to maintain or enhance in the future.

3.3 PLANNING AND SCHEDULING

PERT CHART:



GANTT CHART:



ID	Task Name	Start	End	Duration
1	Research	2025-10-27	2025-11-07	10 days
2	Design	2025-11-07	2025-11-20	10 days
3	Coding	2025-11-21	2026-01-09	36 days
4	Testing	2026-01-09	2026-02-06	21 days
5	Deployment and Maintenance	2026-02-09	2026-02-18	8 days

3.4- SOFTWARE AND HARDWARE REQUIREMENTS:

1] SOFTWARE:

❑ Frontend Technologies:

- React
- HTML
- CSS / Tailwind CSS / Material UI

❑ Backend Technologies:

- Node.js
- Express.js

❑ Database Technology:

- Mongo DB

❑ Authentication

- Firebase

2] HARDWARE:

Development Workstations:

- Processor: Intel i5 or higher
- RAM: Minimum 8 GB, recommended 16 GB
- Storage: SSD with at least 256 GB
- Graphics: Integrated or dedicated GPU

Server Requirements:

- Processor: Multi-core
- RAM: Minimum 16 GB, recommended 32 GB
- Storage: SSD with at least 512 GB
- Network: High-speed internet connection

3.5- PRELIMINARY PRODUCT DESCRIPTION:

- **Secure Authentication:** Role-based login using Firebase Authentication for Receivers, Donors, NGOs, and Admin with controlled access permissions.
- **Emergency Request Portal:** Allows receivers to create, track, and manage urgent requests for resources such as blood, oxygen, plasma, or medicines.
- **Donor Dashboard:** Donors can update availability status, receive real-time emergency alerts, and accept or decline requests based on convenience.
- **Verification System:** NGOs and Admins can verify emergency requests and user authenticity to prevent fraudulent activities and ensure reliability.
- **AI-Based Donor Matching:** Intelligent matching system that identifies the nearest suitable donors based on location, urgency level, and availability.
- **Notification Center:** Sends real-time alerts for new emergency requests, verification updates, donor acceptance, and request completion using Firebase Cloud Messaging.
- **Request Lifecycle Tracking:** Enables users to track request status (Pending, Verified, In Progress, Completed) transparently.
- **Admin Panel:** Admins can monitor system activity, approve or block users, review verification logs, and manage overall system operations.
- **Responsive Interface:** Optimized for accessibility across mobile devices, tablets, and desktops to ensure usability during emergencies.
- **Real-Time Synchronization:** All updates related to requests, donor responses, and verification statuses are reflected instantly using Firebase Firestore.
- **Geolocation Integration:** Map-based visualization using Leaflet.js or Google Maps API to display nearby donors and emergency locations.
- **User-Friendly Interface:** Designed with a clean, intuitive layout to ensure quick navigation and ease of use during high-pressure emergency situations.

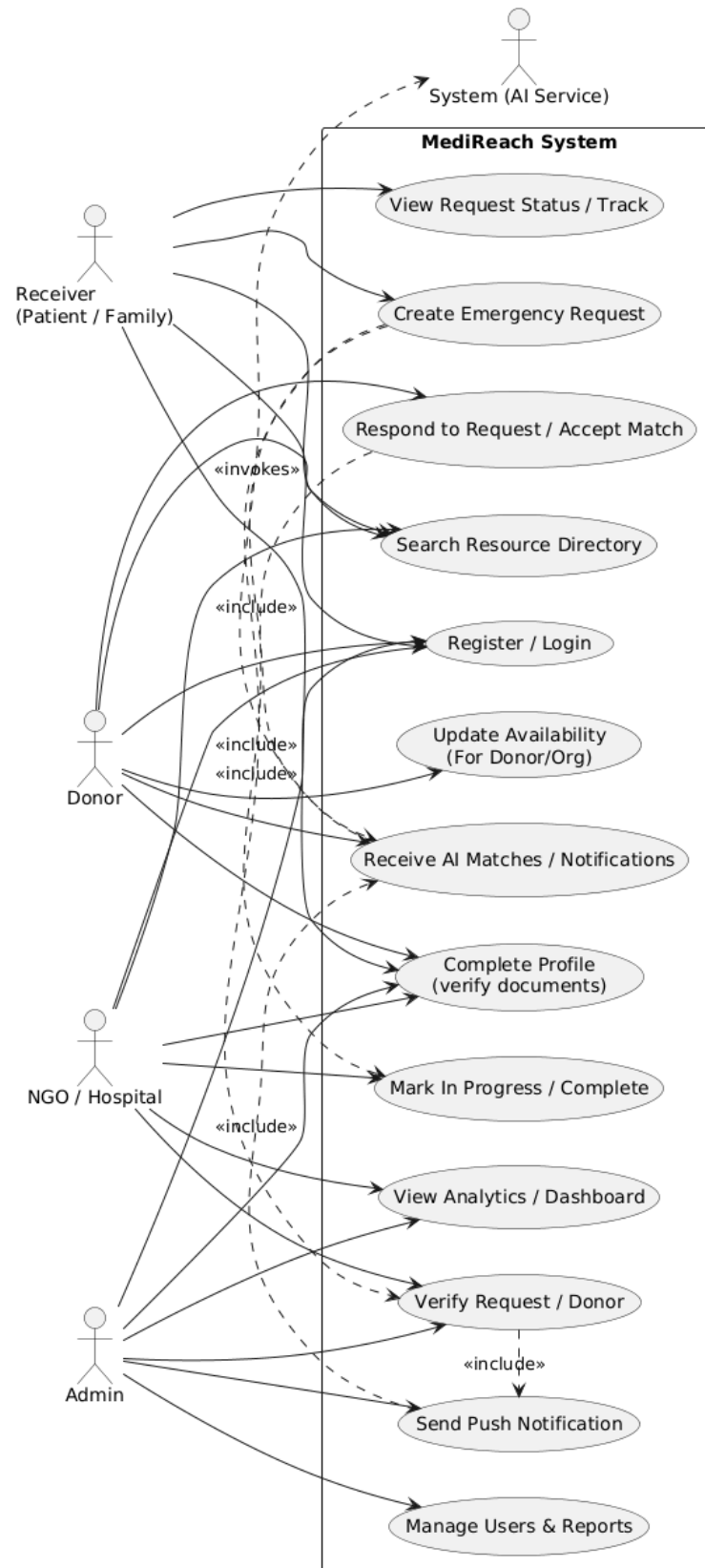
3.6- CONCEPTUAL MODELS:

3.6.1 EVENT TABLE:

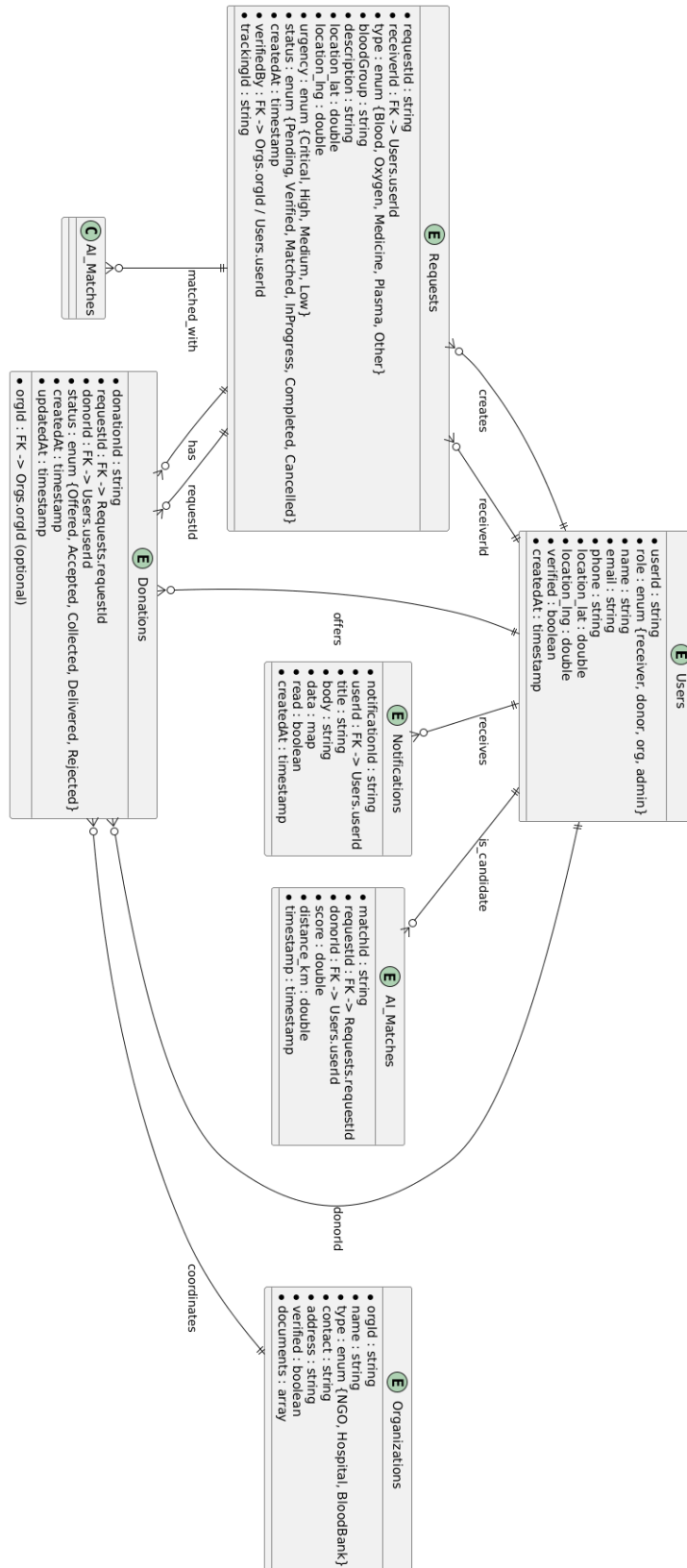
Event Name	Actor	Trigger	System Response	Affected Modules
User Registration/Login	Patient / Doctor / Insurer	Submits login or signup form	Authenticates user using Firebase; grants role-based access	Authentication
Upload Health Record	Doctor	Uploads lab report or prescription	Saves record in Firestore; notifies patient	Data Upload, Notifications
View Health Records	Patient / Doctor / Insurer	Opens "My Records" or "Patient History" page	Fetches and displays records from Firestore	Data Retrieval
Submit Symptom Tracker Form	Patient	Fills and submits weekly health update	Stores symptom data; updates health score; notifies doctor if flagged	Symptom Tracker, Notifications
Track Insurance Claim	Patient / Insurer	Views claim status page	Displays current claim status from Firestore	Claim Tracking, Data Retrieval
Update Claim Status	Insurer	Approves or rejects a claim	Updates claim document in Firestore; notifies patient	Claim Tracking, Notifications
Receive Notification	All Users	Backend triggers event (upload, claim, reminder)	Push notification via Firebase Cloud Messaging	Notifications
Logout	Any user	Clicks logout button	Clears session and redirects to login page	Authentication
Edit Profile Info	Patient / Doctor / Insurer	Updates profile fields	Saves updated user info to Firestore	User Management
Forgot Password	Any user	Requests password reset	Sends password reset link via Firebase	Authentication
Upload Medical Report (PDF/Image)	Doctor / Lab Technician	Uploads report to patient's profile	Stores file in Firebase Storage; updates record metadata	File Upload, Data Management
Event Name	Actor	Trigger	System Response	Affected Modules
User Registration/Login	Patient / Doctor / Insurer	Submits login or signup form	Authenticates user using Firebase; grants role-based access	Authentication

Upload Health Record	Doctor	Uploads lab report or prescription	Saves record in Firestore; notifies patient	Data Upload, Notifications
View Health Records	Patient / Doctor / Insurer	Opens "My Records" or "Patient History" page	Fetches and displays records from Firestore	Data Retrieval
Submit Symptom Tracker Form	Patient	Fills and submits weekly health update	Stores symptom data; updates health score; notifies doctor if flagged	Symptom Tracker, Notifications
Track Insurance Claim	Patient / Insurer	Views claim status page	Displays current claim status from Firestore	Claim Tracking, Data Retrieval
Update Claim Status	Insurer	Approves or rejects a claim	Updates claim document in Firestore; notifies patient	Claim Tracking, Notifications

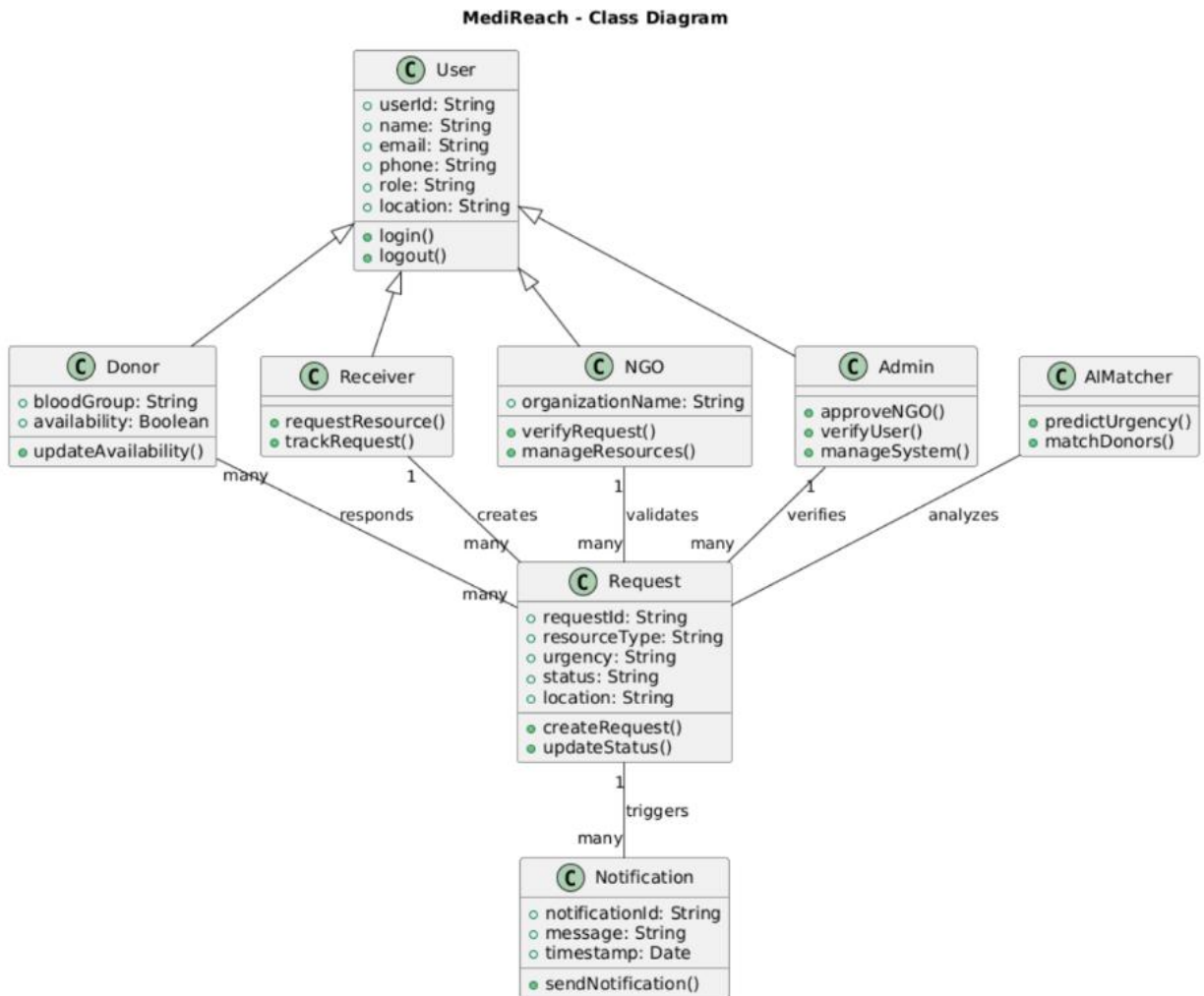
3.6.2 USE CASE DIAGRAM



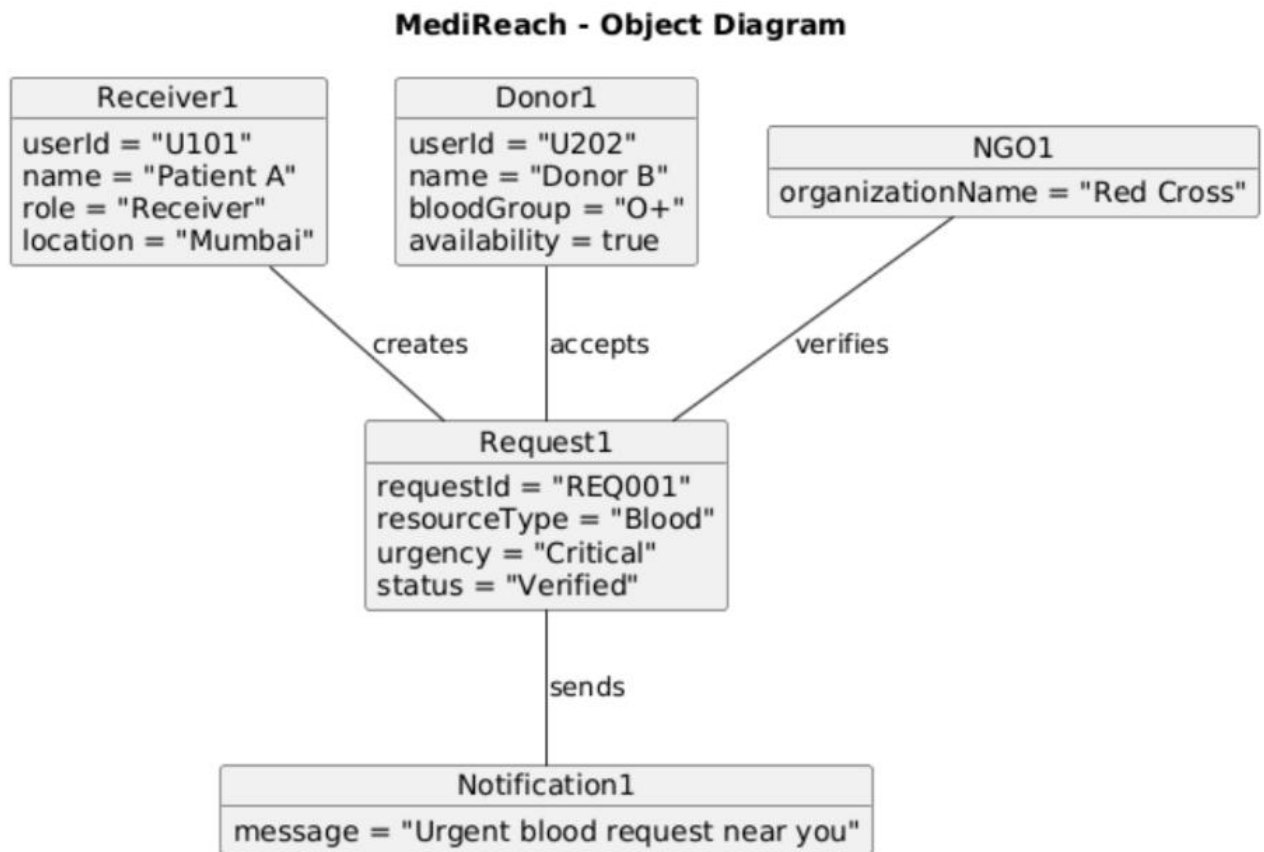
3.6.3 ENTITY RELATIONSHIP DIAGRAM



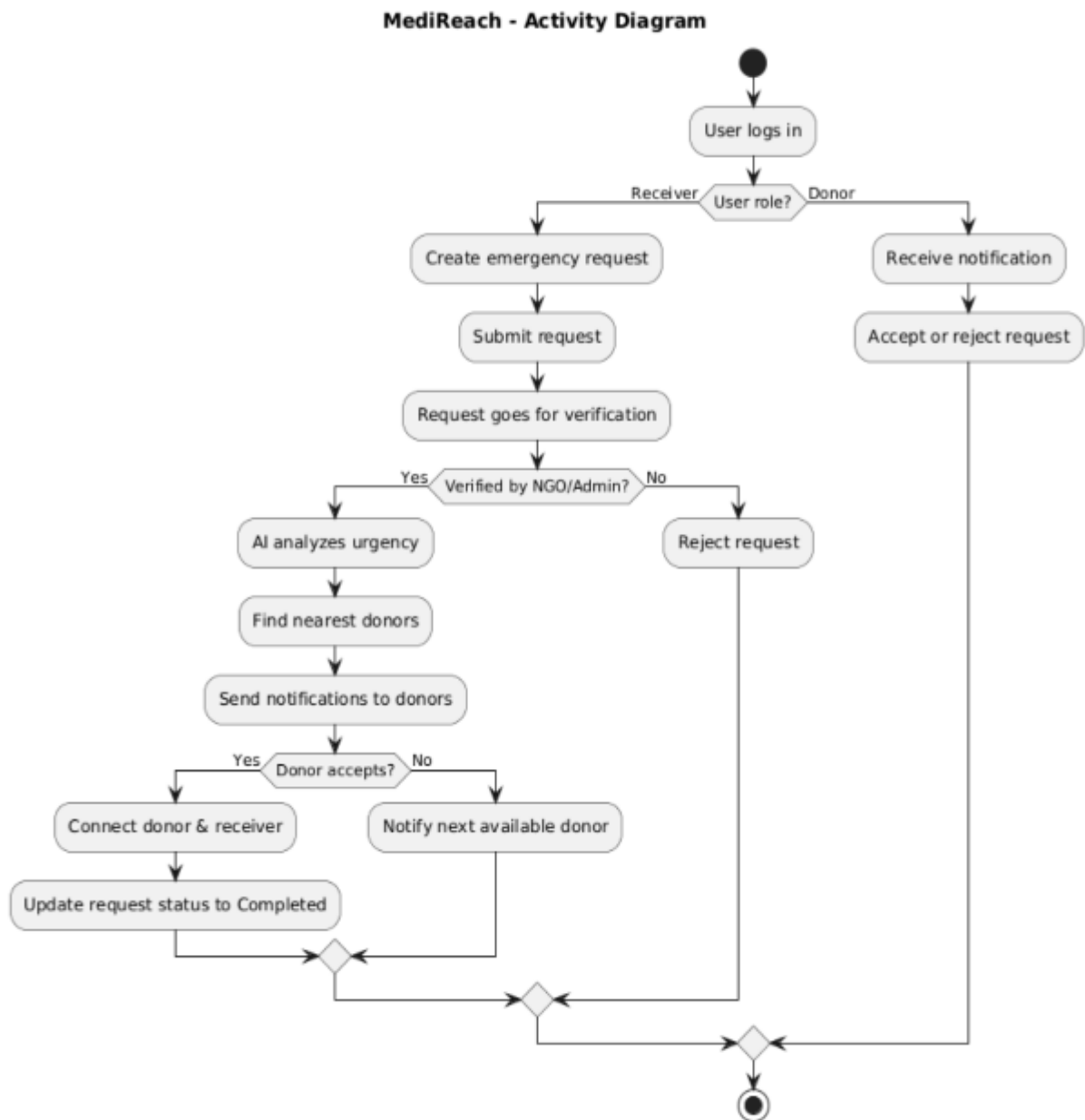
3.6.4 CLASS DIAGRAM:



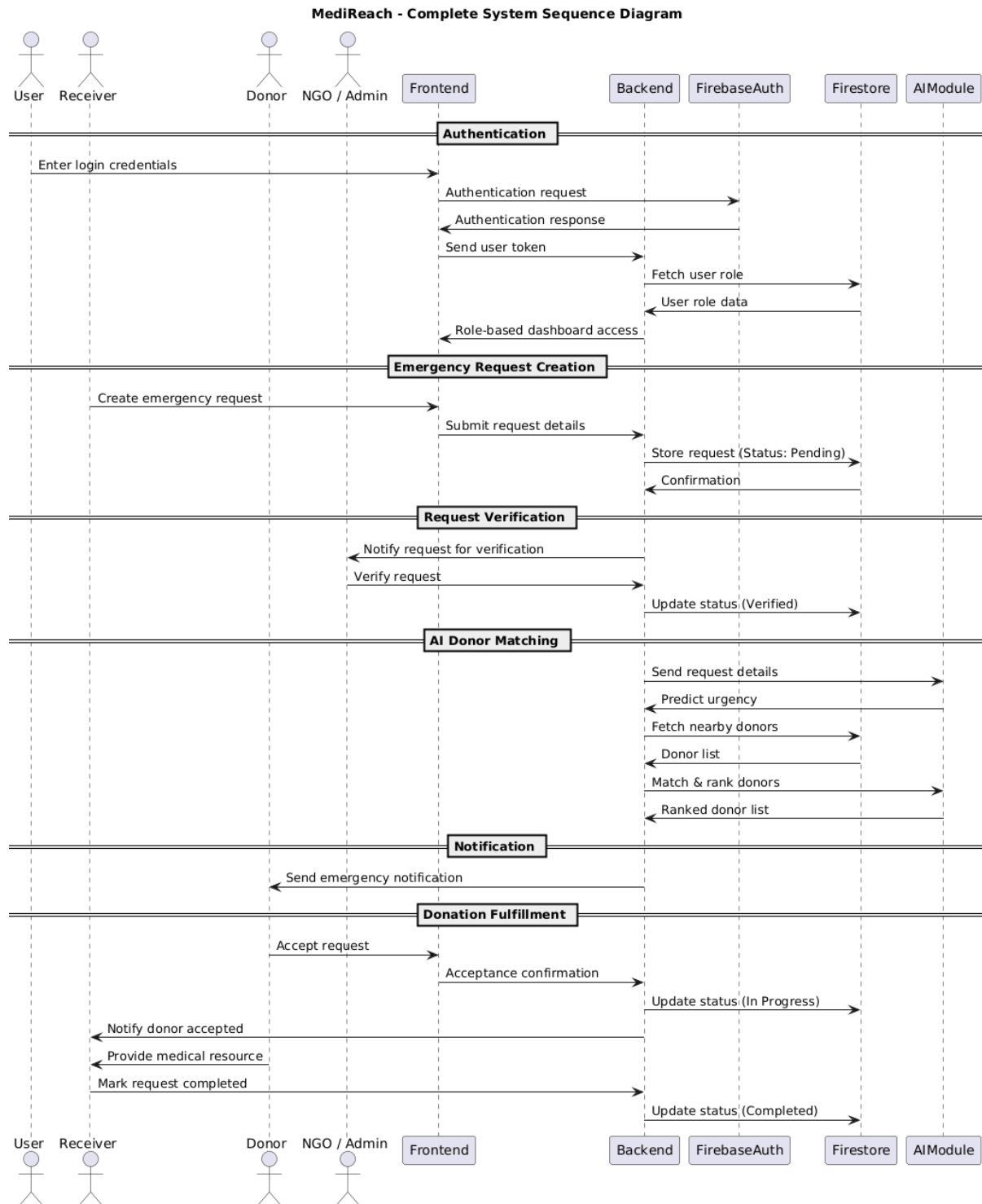
3.6.5 OBJECT DIAGRAM:



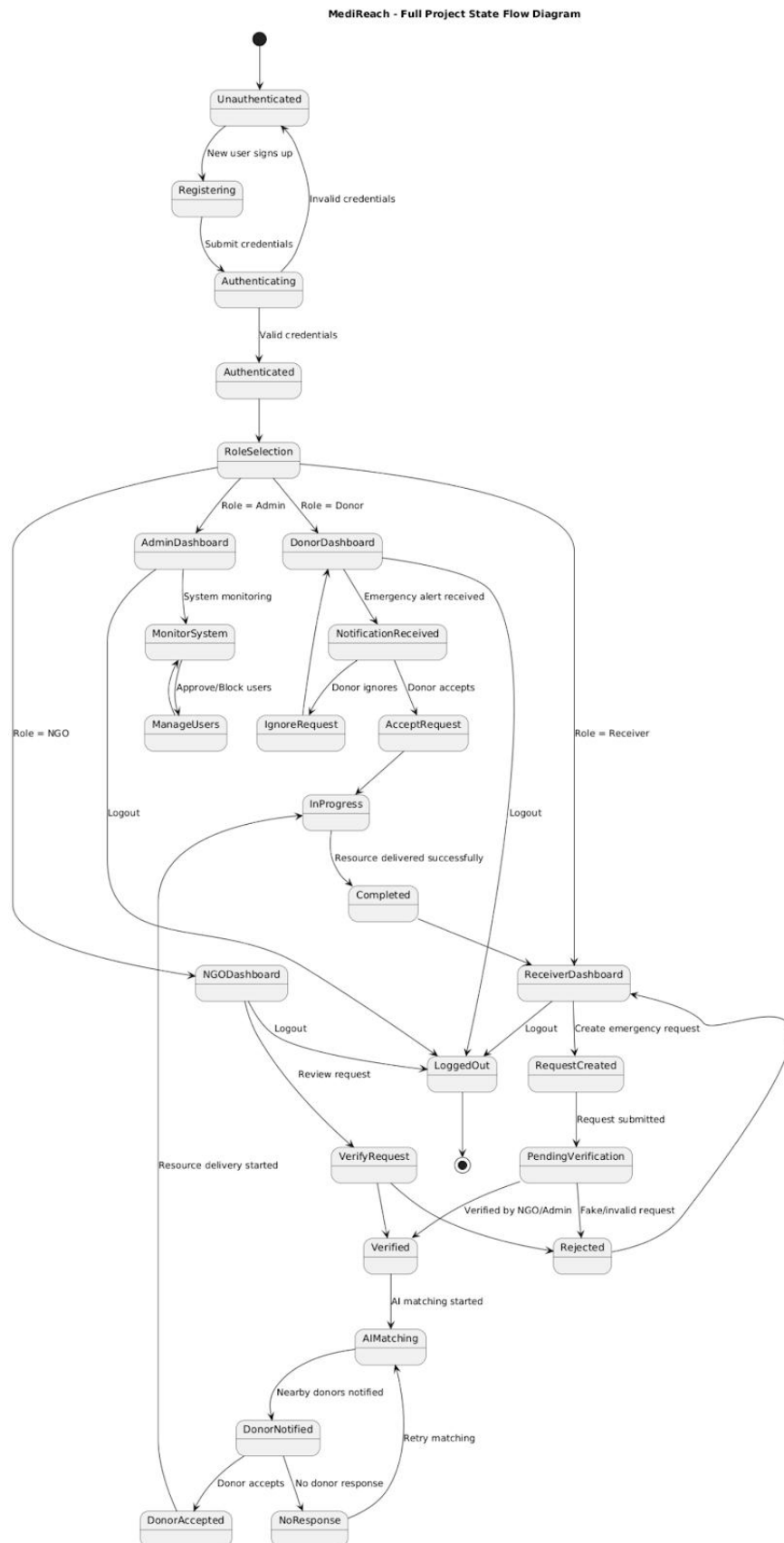
3.6.6 ACTIVITY DIAGRAM:



3.6.7 SEQUENCE DIAGRAM:

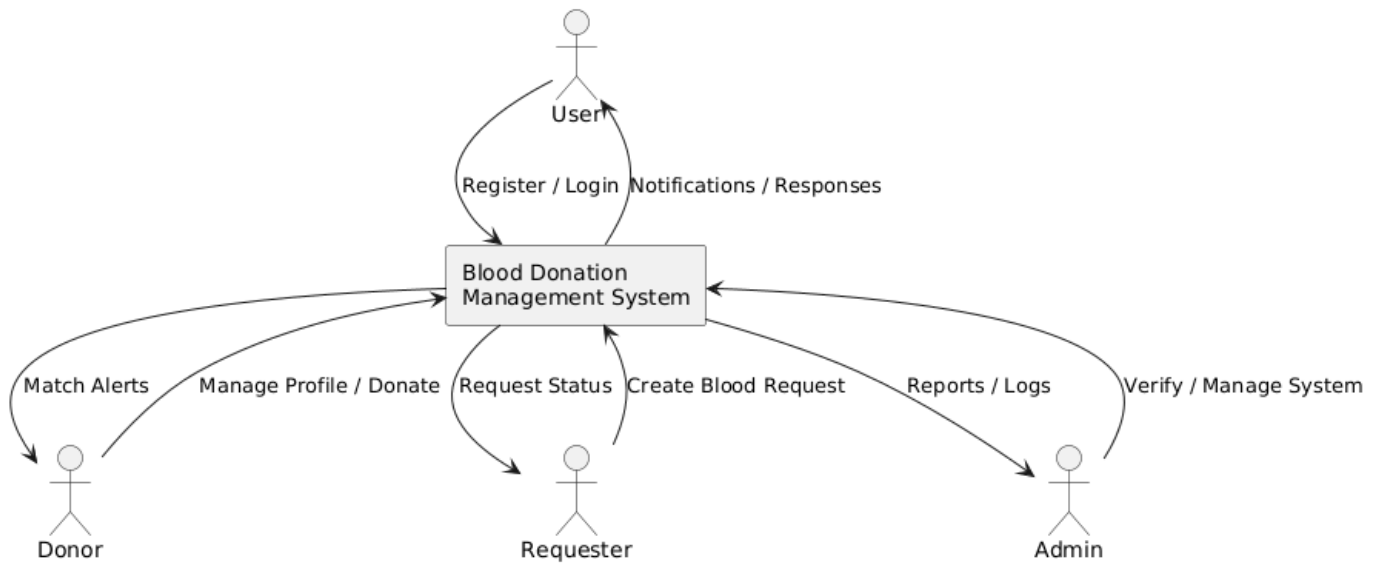


3.6.8 STATE FLOW DIAGRAM:

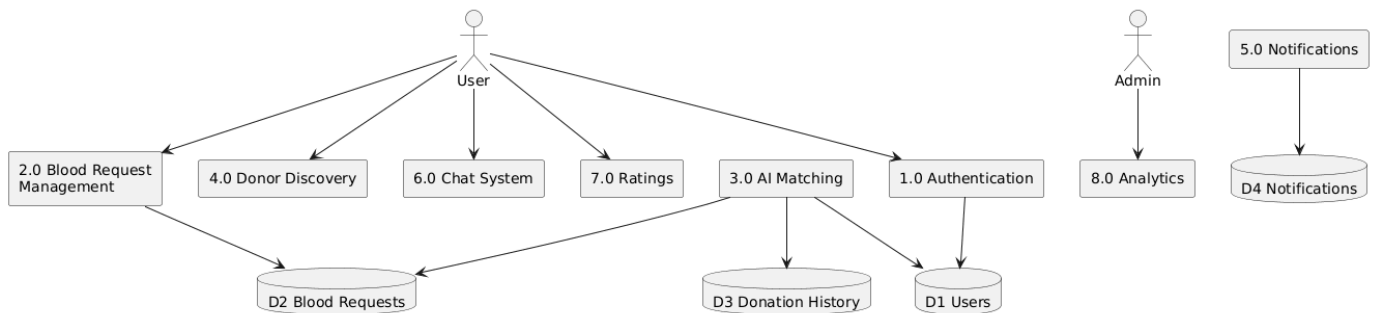


3.6.9 CONTEXT DIAGRAM:

Level 0 DFD - Blood Donation Management System

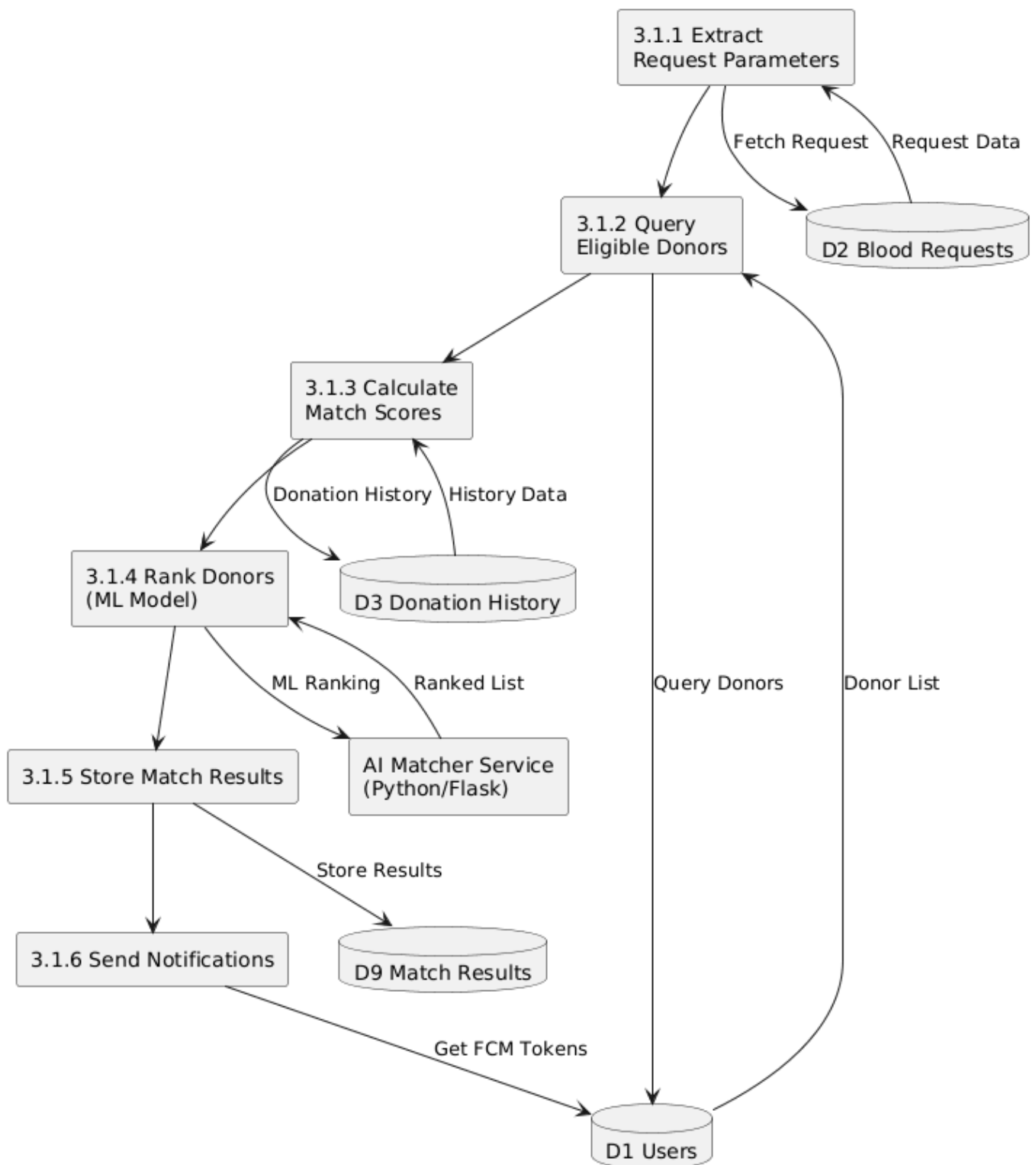


Level 1 DFD - Major Subsystems

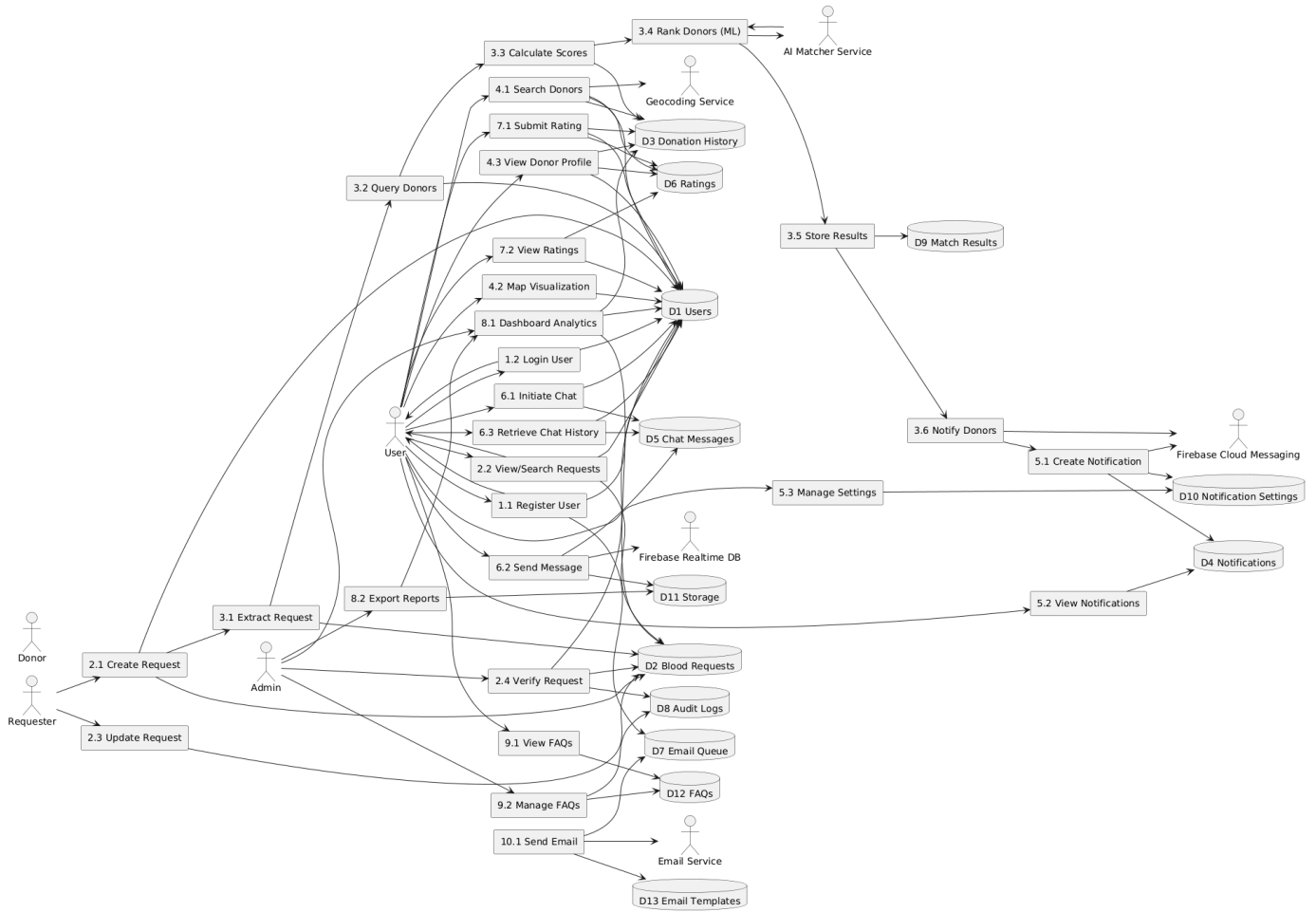


3.6.10 DATA FLOW DIAGRAM:

Level 3 DFD - 3.1 AI Based Donor Matching

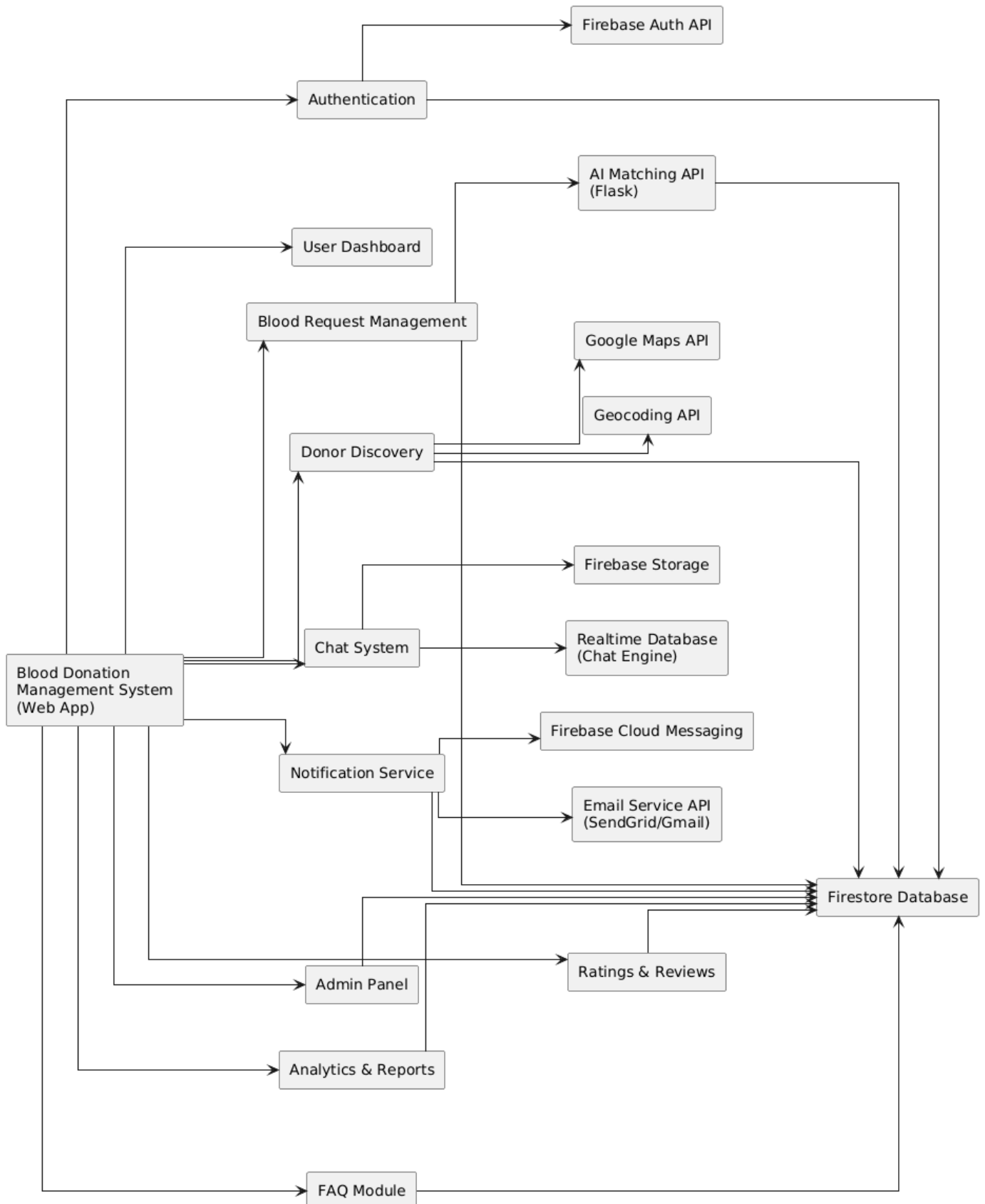


Blood Donation Management System - Complete Level 3 DFD



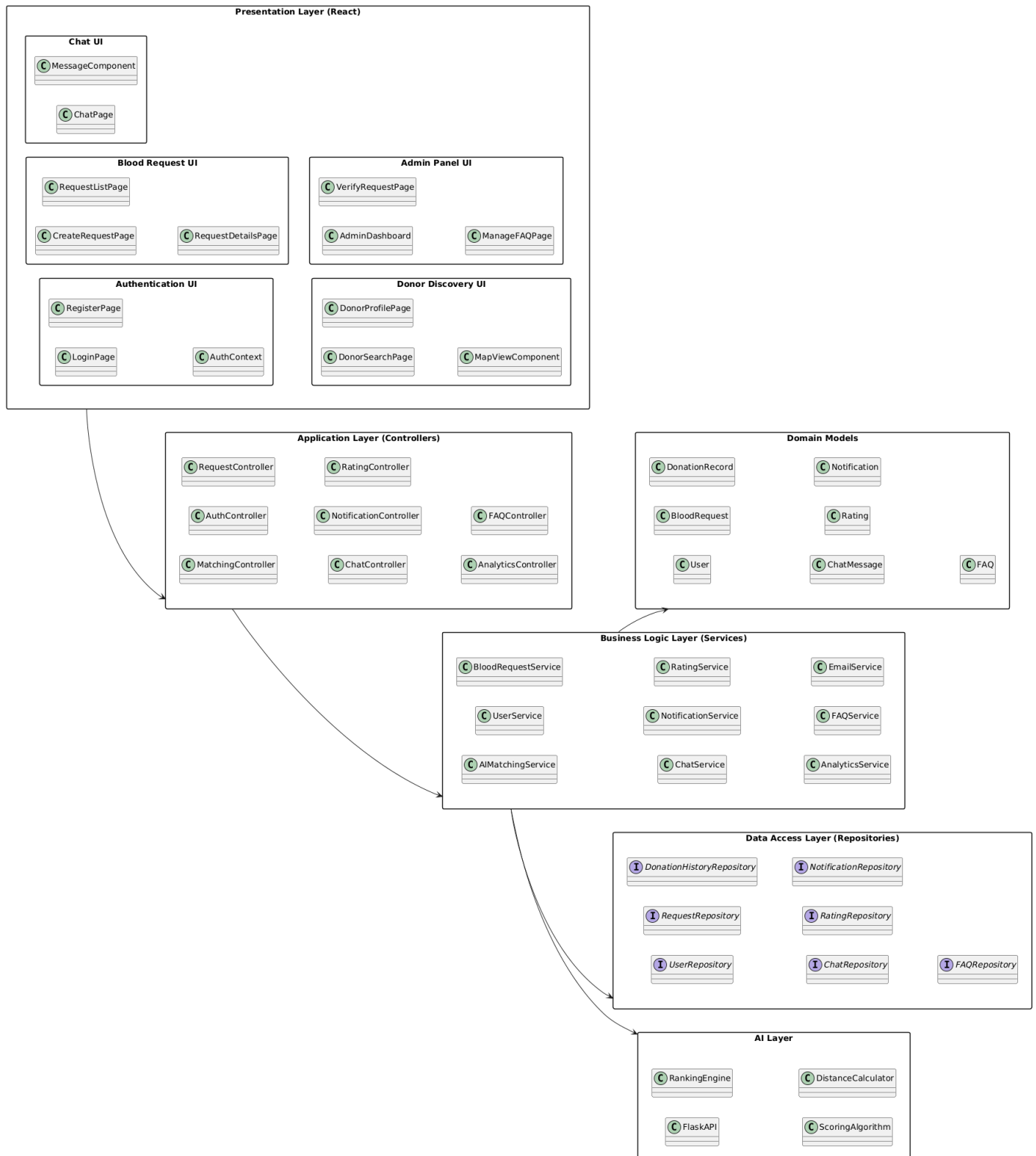
3.6.11 COMPONENT DIAGRAM:

Blood Donation Management System - Structured Component Diagram

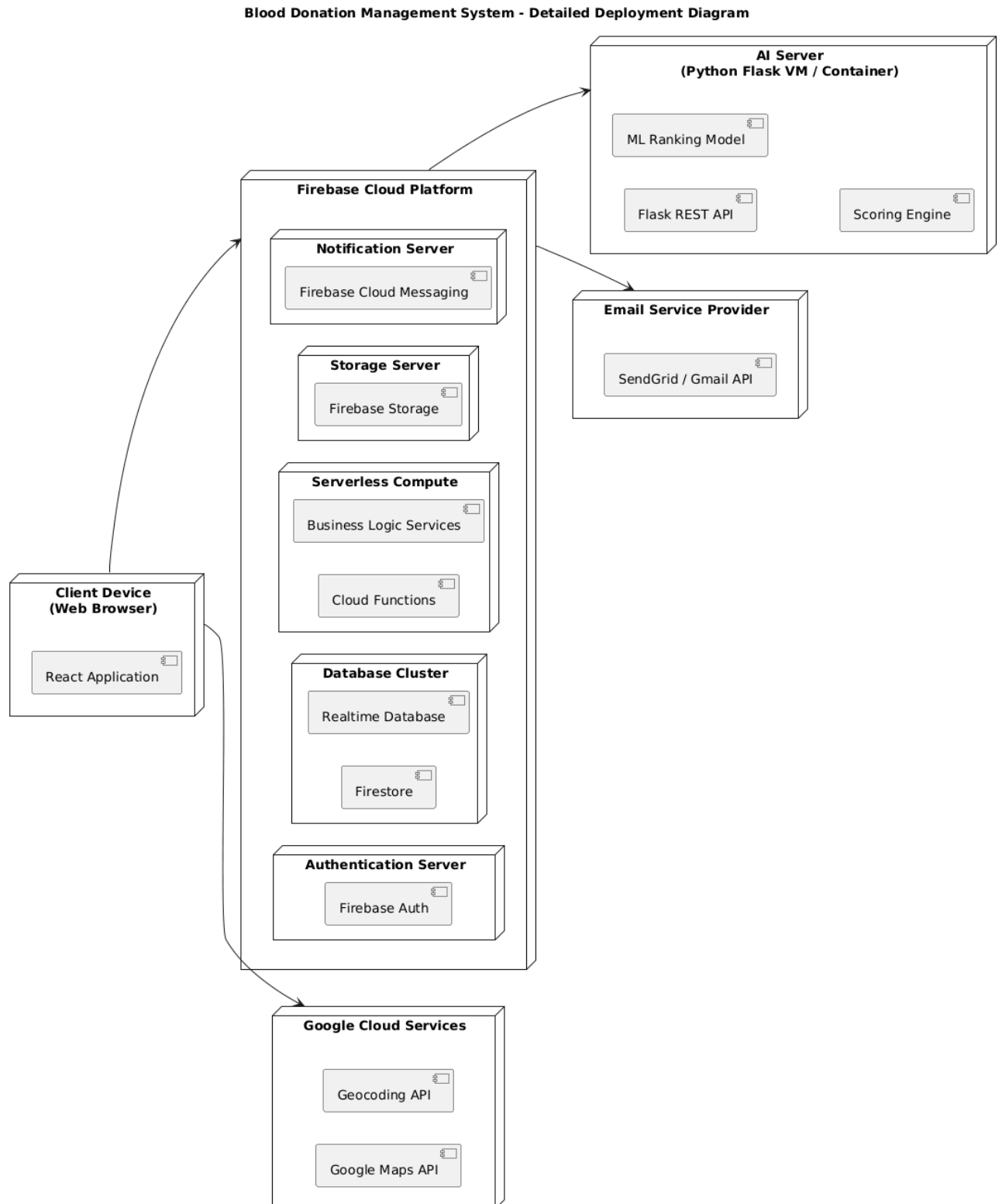


3.6.12 PACKAGE DIAGRAM:

Blood Donation Management System - Detailed Package Structure



3.6.13 DEPLOYMENT DIAGRAM:



4. SYSTEM CODING:

4.1 CODE:

App.js: The main and the entry point of the application and the navigation.

```
App.js
import React from 'react'.
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from './context/AuthContext';
import Navbar from './components/Layout/Navbar';
import Home from './components/Home';
import Login from './components/Auth/Login';
import Register from './components/Auth/Register';
import Dashboard from './components/Dashboard/Dashboard';
import ProtectedRoute from './components/ProtectedRoute';
import CreateRequest from './components/Requests/CreateRequest';
import RequestsList from './components/Requests/RequestsList';
import RequestDetails from './components/Requests/RequestDetails';
import EditRequest from './components/Requests/EditRequest';
import DonorDirectory from './components/Donors/DonorDirectory';
import DonorProfile from './components/Donors/DonorProfile';
import Notifications from './components/Notifications/Notifications';
import NotificationSettings from './components/Notifications/NotificationSettings';
import VerifyRequests from './components/Requests/VerifyRequests';
import Analytics from './components/Analytics/Analytics';
import Chat from './components/Chat/Chat';
import DonationHistory from './components/Donations/DonationHistory';
import Ratings from './components/Ratings/Ratings';
import AdminPanel from './components/Admin/AdminPanel';
import { AboutPage, BlogPage, FAQPage, SupportPage, PartnersPage, PrivacyPage, TermsPage,
CookiesPage } from './components/PlaceholderPages';
import './App.css';

function App() {
  return (
    <Router>
      <AuthProvider>
        <div className="App min-h-screen bg-gray-50">
          <Navbar />
          <Routes>
            { /* Public Routes */ }
            <Route path="/" element={ <Home /> } />
            <Route path="/login" element={ <Login /> } />
            <Route path="/register" element={ <Register /> } />

            { /* Protected Routes */ }
            <Route
              path="/dashboard"
              element={
                <ProtectedRoute>
                  <Dashboard />
                </ProtectedRoute>
              }
            />
          </Routes>
        </div>
      </AuthProvider>
    </Router>
  );
}
```

```

/>

{/* Placeholder routes - to be implemented */}
<Route
  path="/requests"
  element={
    <ProtectedRoute>
      <RequestsList />
    </ProtectedRoute>
  }
/>
<Route
  path="/requests/:id"
  element={
    <ProtectedRoute>
      <RequestDetails />
    </ProtectedRoute>
  }
/>
<Route
  path="/requests/edit/:requestId"
  element={
    <ProtectedRoute allowedRoles={["receiver"]} >
      <EditRequest />
    </ProtectedRoute>
  }
/>
<Route
  path="/donors"
  element={
    <ProtectedRoute>
      <DonorDirectory />
    </ProtectedRoute>
  }
/>
<Route
  path="/donor-profile"
  element={
    <ProtectedRoute allowedRoles={["donor"]} >
      <DonorProfile />
    </ProtectedRoute>
  }
/>
<Route
  path="/create-request"
  element={
    <ProtectedRoute allowedRoles={["receiver"]} >
      <CreateRequest />
    </ProtectedRoute>
  }
/>
<Route
  path="/verify-requests"
  element={
    <ProtectedRoute allowedRoles={["admin", "ngo", "hospital"]} >

```

```

        <VerifyRequests />
      </ProtectedRoute>
    }
  />
  <Route
    path="/notifications"
    element={
      <ProtectedRoute>
        <Notifications />
      </ProtectedRoute>
    }
  />
  <Route
    path="/notification-settings"
    element={
      <ProtectedRoute>
        <NotificationSettings />
      </ProtectedRoute>
    }
  />
  <Route
    path="/analytics"
    element={
      <ProtectedRoute allowedRoles={['admin', 'ngo', 'hospital']}>
        <Analytics />
      </ProtectedRoute>
    }
  />
  <Route
    path="/chat"
    element={
      <ProtectedRoute>
        <Chat />
      </ProtectedRoute>
    }
  />
  <Route
    path="/donation-history"
    element={
      <ProtectedRoute allowedRoles={['donor']}>
        <DonationHistory />
      </ProtectedRoute>
    }
  />
  <Route
    path="/ratings"
    element={
      <ProtectedRoute>
        <Ratings />
      </ProtectedRoute>
    }
  />
  <Route
    path="/admin"
    element={

```

```

        <ProtectedRoute allowedRoles={['admin']}>
          <AdminPanel />
        </ProtectedRoute>
      }
    />

    { /* Footer Placeholder Pages */ }
    <Route path="/about" element={ <AboutPage /> } />
    <Route path="/blog" element={ <BlogPage /> } />
    <Route path="/faq" element={ <FAQPage /> } />
    <Route path="/support" element={ <SupportPage /> } />
    <Route path="/partners" element={ <PartnersPage /> } />
    <Route path="/privacy" element={ <PrivacyPage /> } />
    <Route path="/terms" element={ <TermsPage /> } />
    <Route path="/cookies" element={ <CookiesPage /> } />
    <Route path="/how-it-works" element={ <AboutPage /> } />
    <Route path="/success-stories" element={ <BlogPage /> } />

    { /* Catch all */ }
    <Route path="*" element={ <Navigate to="/" /> } />
  </Routes>
</div>
</AuthProvider>
</Router>
);
}

```

export default App;

TrainDonorModel.py

```

import pandas as pd
import joblib
import os
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, classification_report, confusion_matrix
)
import numpy as np

print("=" * 60)
print("MediReach Donor Matching Model Training")
print("=" * 60)

# Set paths
DATA_PATH = os.path.join(os.path.dirname(__file__), '..', 'data', 'donor_training_data.csv')
MODEL_PATH = os.path.join(os.path.dirname(__file__), 'donor_model.pkl')
FEATURES_PATH = os.path.join(os.path.dirname(__file__), 'feature_columns.pkl')

print(f"\n📁 Loading data from: {DATA_PATH}")

# Load data
try:

```

```

df = pd.read_csv(DATA_PATH)
print(f"✓ Data loaded successfully: {df.shape[0]} samples, {df.shape[1]} columns")
except FileNotFoundError:
    print(f"✗ Error: Dataset not found at {DATA_PATH}")
    print("Please ensure donor_training_data.csv exists in the data/ directory")
    exit(1)

# Display dataset info
print("\n📊 Dataset Overview:")
print(df.head())
print("\n📊 Target Distribution:")
print(df['successful'].value_counts())
print(f"\nSuccess Rate: {df['successful'].mean():.2%}")

# Separate features and target
X = df.drop('successful', axis=1)
y = df['successful']

feature_names = list(X.columns)
print(f"\nFeatures ( {len(feature_names)} ):")
for i, feature in enumerate(feature_names, 1):
    print(f" {i}. {feature}")

# Split data for training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"\n🔪 Data Split:")
print(f" Training set: {X_train.shape[0]} samples")
print(f" Test set: {X_test.shape[0]} samples")

# Initialize XGBoost model
print("\n🔧 Initializing XGBoost Classifier...")
model = XGBClassifier(
    n_estimators=200,      # Number of trees
    max_depth=5,          # Maximum tree depth
    learning_rate=0.1,     # Learning rate
    subsample=0.8,        # Subsample ratio of training instances
    colsample_bytree=0.8,  # Subsample ratio of features
    eval_metric='logloss', # Evaluation metric
    random_state=42,
    use_label_encoder=False
)

# Train model
print("🔧 Training model...")
model.fit(
    X_train, y_train,
    eval_set=[(X_test, y_test)],
    verbose=False
)
print("✓ Training completed!")

```

```

# Cross-validation
print("\n🔍 Performing 5-fold cross-validation...")
cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print(f"CV Accuracy: {cv_scores.mean():.4f} (+/- {cv_scores.std() * 2:.4f})")

# Make predictions
print("\n🔮 Making predictions on test set...")
y_pred = model.predict(X_test)
y_pred_proba = model.predict_proba(X_test)[: , 1]

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred_proba)

print("\n📊 Model Performance Metrics:")
print(f" Accuracy: {accuracy:.4f}")
print(f" Precision: {precision:.4f}")
print(f" Recall: {recall:.4f}")
print(f" F1 Score: {f1:.4f}")
print(f" ROC AUC: {roc_auc:.4f}")

print("\n📋 Detailed Classification Report:")
print(classification_report(y_test, y_pred, target_names=['Failed', 'Successful']))

print("\n📊 Confusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)
print(f"True Negatives: {cm[0][0]}")
print(f"False Positives: {cm[0][1]}")
print(f"False Negatives: {cm[1][0]}")
print(f"True Positives: {cm[1][1]}")

# Feature importance
print("\n★ Feature Importance:")
feature_importance = pd.DataFrame({
    'feature': feature_names,
    'importance': model.feature_importances_
}).sort_values('importance', ascending=False)

for idx, row in feature_importance.iterrows():
    print(f" {row['feature']}:25s} : {row['importance']:.4f}")

# Save model
print(f"\n💾 Saving model to: {MODEL_PATH}")
joblib.dump(model, MODEL_PATH)
print("✓ Model saved successfully!")

print(f"\n💾 Saving feature names to: {FEATURES_PATH}")
joblib.dump(feature_names, FEATURES_PATH)
print("✓ Feature names saved successfully!")

```

```

# Model summary
print("\n" + "=" * 60)
print("☑ TRAINING COMPLETE")
print("=" * 60)
print(f'Model Type: XGBoost Classifier')
print(f'Training Samples: {len(X_train)}')
print(f'Test Samples: {len(X_test)}')
print(f'Features: {len(feature_names)}')
print(f'Best Accuracy: {accuracy:.4f}')
print(f'Best F1 Score: {f1:.4f}')
print("\n ♀ Next Steps:")
print(" 1. Review model performance metrics")
print(" 2. Test inference with: python test_donor_model.py")
print(" 3. Deploy to Flask app for production use")
print(" 4. Monitor prediction performance in production")
print("=" * 60)

```

TrainUrgencyModel.py

```

import pandas as pd
import joblib
import os
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix
)
import numpy as np

print("=" * 60)
print("MediReach Urgency Prediction Model Training")
print("=" * 60)

# Set paths
DATA_PATH = os.path.join(os.path.dirname(__file__), '..', 'data', 'urgency_training_data.csv')
MODEL_PATH = os.path.join(os.path.dirname(__file__), 'urgency_model.pkl')

print(f'\n 📁 Loading data from: {DATA_PATH}')

# Load data
try:
    df = pd.read_csv(DATA_PATH)
    print(f'✓ Data loaded successfully: {df.shape[0]} samples, {df.shape[1]} columns')
except FileNotFoundError:
    print(f'✗ Error: Dataset not found at {DATA_PATH}')
    print("Please ensure urgency_training_data.csv exists in the data/ directory")
    exit(1)

# Display dataset info

```

```

print("\n📊 Dataset Overview:")
print(df.head())
print("\n📈 Urgency Label Distribution:")
print(df['urgency_label'].value_counts().sort_index())

# Separate features and target
X = df['description']
y = df['urgency_label']

# Display label mapping
urgency_labels = sorted(y.unique())
print(f"\n🏷️ Urgency Labels ({len(urgency_labels)}):")
for i, label in enumerate(urgency_labels):
    count = (y == label).sum()
    percentage = count / len(y) * 100
    print(f" {i+1}. {label:10s} : {count:3d} samples ({percentage:5.1f}%)")

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"\n📊 Data Split:")
print(f" Training set: {X_train.shape[0]} samples")
print(f" Test set: {X_test.shape[0]} samples")

# Create ML Pipeline
print("\n🏗️ Building ML Pipeline...")
print(" 1. TF-IDF Vectorizer (max_features=5000)")
print(" 2. Logistic Regression (multi-class)")

pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        max_features=5000,    # Maximum number of features
        ngram_range=(1, 2),  # Use unigrams and bigrams
        min_df=1,            # Minimum document frequency (1 for small datasets)
        max_df=1.0,          # Maximum document frequency (1.0 = no upper limit for small
datasets)
        lowercase=True,
        stop_words='english'
    )),
    ('clf', LogisticRegression(
        max_iter=1000,
        multi_class='multinomial',
        solver='lbfgs',
        random_state=42,
        C=1.0                # Regularization strength
    ))
])

```

```

# Train model
print("\n🎯 Training model...")
pipeline.fit(X_train, y_train)
print("✓ Training completed!")

# Cross-validation
print("\n🔄 Performing 5-fold cross-validation...")
# Convert Series to numpy array for type compatibility with cross_val_score
cv_scores = cross_val_score(pipeline, X.to_numpy(), y, cv=5, scoring='accuracy')
print(f"CV Accuracy: {cv_scores.mean():.4f} (+/- {cv_scores.std() * 2:.4f})")

# Make predictions
print("\n🎯 Making predictions on test set...")
y_pred = pipeline.predict(X_test)
y_pred_proba = pipeline.predict_proba(X_test)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

print("\n📊 Model Performance Metrics:")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f} (weighted)")
print(f"Recall: {recall:.4f} (weighted)")
print(f"F1 Score: {f1:.4f} (weighted)")

print("\n📋 Detailed Classification Report:")
print(classification_report(y_test, y_pred, target_names=urgency_labels))

print("\n📊 Confusion Matrix:")
cm = confusion_matrix(y_test, y_pred, labels=urgency_labels)
print("\n" + " " * 12 + " ".join([f" {label:8s}" for label in urgency_labels]))
for i, label in enumerate(urgency_labels):
    print(f" {label:10s} : " + " ".join([f" {cm[i][j]:8d}" for j in range(len(urgency_labels))]))

# Per-class accuracy
print("\n📊 Per-Class Accuracy:")
for i, label in enumerate(urgency_labels):
    correct = cm[i][i]
    total = cm[i].sum()
    acc = correct / total if total > 0 else 0
    print(f" {label:10s} : {acc:.4f} ({correct}/{total})")

# Feature importance (top TF-IDF features per class)
print("\n⭐ Top Features per Urgency Class:")
vectorizer = pipeline.named_steps['tfidf']
classifier = pipeline.named_steps['clf']

```

```

feature_names = vectorizer.get_feature_names_out()

for i, label in enumerate(urgency_labels):
    print(f"\n {label}:")
    # Get coefficients for this class
    coef = classifier.coef_[i]
    top_indices = np.argsort(coef)[-10:][::-1]
    for idx in top_indices:
        print(f"   - {feature_names[idx]:20s} : {coef[idx]:7.4f}")

# Test predictions on sample descriptions
print("\n 📄 Sample Predictions:")
test_samples = [
    "Emergency bleeding patient needs blood immediately",
    "Urgent surgery scheduled for tomorrow",
    "Routine transfusion needed for regular patient",
    "Critical accident case hemorrhaging severely"
]

for sample in test_samples:
    pred = pipeline.predict([sample])[0]
    proba = pipeline.predict_proba([sample])[0]
    confidence = max(proba)
    print(f"\n Text: '{sample[:50]}...")
    print(f" Prediction: {pred} (confidence: {confidence:.3f})")

# Save model
print(f"\n 💾 Saving model to: {MODEL_PATH}")
joblib.dump(pipeline, MODEL_PATH)
print("✓ Model saved successfully!")

# Model summary
print("\n" + "=" * 60)
print("✅ TRAINING COMPLETE")
print("=" * 60)
print(f"Model Type: TF-IDF + Logistic Regression")
print(f"Training Samples: {len(X_train)}")
print(f"Test Samples: {len(X_test)}")
print(f"Urgency Classes: {len(urgency_labels)}")
print(f"Best Accuracy: {accuracy:.4f}")
print(f"Best F1 Score: {f1:.4f}")
print("\n 📌 Next Steps:")
print(" 1. Review model performance metrics")
print(" 2. Test inference with: python test_urgency_model.py")
print(" 3. Deploy to Flask app for production use")
print(" 4. Monitor prediction accuracy in production")
print("=" * 60)

```

PreProces.py

```

"""
Preprocessing Module for MediReach ML System
Handles feature engineering and data transformation
"""

import numpy as np
from geopy.distance import geodesic

def encode_urgency(urgency):
    """
    Encode urgency level to numerical value
    Args:
        urgency (str): Urgency level (Low, Medium, High, Critical)
    Returns:
        int: Encoded value (0-3)
    """
    mapping = {
        "Low": 0,
        "Medium": 1,
        "High": 2,
        "Critical": 3
    }
    return mapping.get(urgency, 0)

def decode_urgency(encoded_value):
    """
    Decode numerical urgency value back to label
    Args:
        encoded_value (int): Numerical urgency (0-3)
    Returns:
        str: Urgency label
    """
    mapping = {
        0: "Low",
        1: "Medium",
        2: "High",
        3: "Critical"
    }
    return mapping.get(encoded_value, "Low")

def calculate_distance(donor_coords, receiver_coords):
    """
    Calculate geodesic distance between two points
    Args:
        donor_coords (dict): {'lat': float, 'lng': float}
        receiver_coords (dict): {'lat': float, 'lng': float}
    Returns:
        float: Distance in kilometers
    """
    try:

```

```

    donor_point = (donor_coords.get('lat'), donor_coords.get('lng'))
    receiver_point = (receiver_coords.get('lat'), receiver_coords.get('lng'))
    return geodesic(donor_point, receiver_point).kilometers
except Exception as e:
    print(f'Distance calculation error: {e}')
    return 50.0 # Default distance

```

```

def prepare_donor_features(donor, request_data):
    """

```

Convert donor + request into ML feature vector

Features:

1. distance_km: Geographic distance
2. is_available: Binary availability status
3. completion_rate: Historical success rate
4. total_donations: Experience metric
5. urgent_response_rate: Urgency performance
6. urgency_encoded: Request urgency level
7. blood_match: Blood type compatibility

Args:

donor (dict): Donor information
 request_data (dict): Blood request information

Returns:

numpy.ndarray: Feature vector [7 features]

"""

Feature 1: Distance

distance = donor.get("distance_km")

if distance is None:

Calculate if not provided

if donor.get("coordinates") and request_data.get("coordinates"):

distance = calculate_distance(donor["coordinates"], request_data["coordinates"])

else:

distance = 50.0 # Default

Feature 2: Availability (binary)

is_available = 1 if donor.get("availability", False) else 0

Feature 3: Completion rate (success ratio)

total = donor.get("totalDonations", 0)

completed = donor.get("completedDonations", 0)

completion_rate = completed / total if total > 0 else 0.5 # Neutral for new donors

Feature 4: Total donations (experience)

total_donations = total

Feature 5: Urgent response rate

urgent_response_rate = donor.get("urgentResponseRate", 0.5)

Feature 6: Urgency encoding

```

urgency_encoded = encode_urgency(request_data.get("urgency", "Low"))

# Feature 7: Blood type match (binary)
blood_match = 1 if donor.get("bloodGroup") == request_data.get("bloodGroup") else 0

# Return feature vector
return np.array([
    distance,
    is_available,
    completion_rate,
    total_donations,
    urgent_response_rate,
    urgency_encoded,
    blood_match
])

def extract_text_features(description, units=1):
    """
    Extract features from text description for urgency prediction
    Args:
        description (str): Request description
        units (int): Number of blood units requested
    Returns:
        dict: Extracted features
    """
    desc_lower = description.lower()

    features = {
        'description': description,
        'units': units,
        'length': len(description),
        'word_count': len(description.split()),
        'has_emergency': int('emergency' in desc_lower),
        'has_critical': int('critical' in desc_lower),
        'has_urgent': int('urgent' in desc_lower),
        'has_immediately': int('immediately' in desc_lower or 'immediate' in desc_lower),
    }

    return features

def get_feature_names():
    """
    Get standardized feature names for donor matching model
    Returns:
        list: Feature names in correct order
    """
    return [
        'distance_km',
        'is_available',
        'completion_rate',
        'total_donations',
    ]

```

```

    'urgent_response_rate',
    'urgency_encoded',
    'blood_match'
]

def validate_donor_features(features):
    """
    Validate and clip feature values to expected ranges
    Args:
        features (np.ndarray): Feature vector
    Returns:
        np.ndarray: Validated feature vector
    """
    validated = features.copy()

    # Distance: 0-500 km
    validated[0] = np.clip(validated[0], 0, 500)

    # is_available: 0 or 1
    validated[1] = np.clip(validated[1], 0, 1)

    # completion_rate: 0-1
    validated[2] = np.clip(validated[2], 0, 1)

    # total_donations: 0-100
    validated[3] = np.clip(validated[3], 0, 100)

    # urgent_response_rate: 0-1
    validated[4] = np.clip(validated[4], 0, 1)

    # urgency_encoded: 0-3
    validated[5] = np.clip(validated[5], 0, 3)

    # blood_match: 0 or 1
    validated[6] = np.clip(validated[6], 0, 1)

    return validated

```

4.2 DATA DICTIONARY:

- **Users Collection**

Data Attribute	Description	Data Type
Name	Full legal name of the user	String
Email	Email address used for authentication and communication	String
Mobile	User's mobile phone number	String
Role	Role of the user (Donor / Receiver / NGO / Admin)	String
Location	City or area of residence	String
BloodGroup	Blood group (for donors)	String
Availability	Availability status of donor	Boolean
UID	Unique identifier assigned by Firebase	String
ProfilePhoto	URL link to profile image stored in Firebase Storage	String (URL)
Verified	Verification status by NGO/Admin	Boolean

- **Emergency Request Collection**

Data Attribute	Description	Data Type
RequestID	Unique emergency request identifier	String
ResourceType	Required resources (Blood, Oxygen, Plasma, Medicine)	String
Urgency	Emergency level (Low / Medium / Critical)	String
Location	Location of the emergency	String
Status	Request status (Pending, Verified, In Progress, Completed)	String
CreatedAt	Timestamp when request was created	Timestamp
ReceiverUID	UID of the receiver	String

- **Donor Response Collection (Sub-collection under Requests)**

Data Attribute	Description	Data Type
DonorUID	UID of responding donor	String
ResponseStatus	Accepted / Rejected	String
ResponseTime	Time of response	Timestamp

- **Verification Logs Collection**

Data Attribute	Description	Data Type
VerifiedBy	NGO/Admin UID	String
VerificationStatus	Approved / Rejected	String

Data Attribute	Description	Data Type
Remarks	Verification comments	String
VerifiedAt	Timestamp of verification	Timestamp

4.3 PROGRAM DESCRIPTION FOR THE PROJECT

1] INTRODUCTION:

□ **Purpose:** The Medi-Reach project aims to develop a secure, real-time, and AI-enabled emergency medical resource platform that connects people in urgent need with verified donors, hospitals, and NGOs. The system ensures authenticity of requests, reduces response time, and eliminates misinformation by using role-based verification and intelligent donor matching.

□ **Key Features:**

- **Secure role-based authentication**
- **Emergency request creation and tracking**
- **NGO/Admin verification system**
- **AI-based donor matching using geolocation**
- **Real-time notifications**
- **Request for lifecycle monitoring**

2] OVERVIEW

□ Medi-Reach is designed to streamline emergency medical assistance by providing a centralized platform for donors, receivers, NGOs, and administrators. The system integrates Firebase for authentication and real-time data synchronization, Node.js for backend processing, and an AI microservice for urgency prediction and donor matching.

□ **Target Audience**

- **Receivers:** Patients or families in medical emergencies
- **Donors:** Individuals willing to donate blood, plasma, or resources
- **NGOs/Hospitals:** Organizations verifying and coordinating requests
- **Admins:** System controllers ensuring authenticity and governance.

3] FUNCTIONALITY

□ **Key Features and Their Working:**

1. User Registration with Role-Based Verification

o **Working:** During signup, users select their role (Donor, Receiver, NGO, Admin). The system verifies email credentials using Firebase Authentication and stores role-based data securely in Firestore. NGOs may require additional document verification before approval.

o **Key UI Components:** Registration form with role selection dropdown and verification confirmation message.

2. Login

- **Working:** Users log in using email and password stored in Firebase Authentication. After successful authentication, the system fetches the user role and redirects to the appropriate dashboard.
 - **Validations:** Ensures correct credentials, displays error messages for invalid login attempts.
3. Emergency Request Creation (Receiver Module)
- **Working:** Receivers submit emergency requests specifying resource type (Blood, Oxygen, Plasma, Medicine), urgency level, and location. The request is stored in Firestore with a “Pending” status until verified by NGO/Admin.
 - **Key UI Components:** Request form, urgency selector, location input field, submit button. Patient Dashboard
4. Donor Availability & Response Module
- **Working:** Donors can update their availability status and receive real-time notifications for verified requests. Donors can accept or decline requests directly from their dashboard.
 - **Key UI Components:** Availability toggle button, notification panel, accept/reject buttons.
5. Verification System (NGO/Admin Module)
- **Working:** NGOs or Admins review pending emergency requests and verify authenticity before approval. Verified requests proceed to AI-based donor matching.
 - **Key UI Components:** Verification dashboard, request details panel, approve/reject buttons.
6. AI-Based Donor Matching
- **Working:** Once verified, the AI module analyzes urgency and location to identify and rank the nearest suitable donors. The system then triggers notifications to matched donors.
 - **Key UI Components:** Donor match list display, urgency indicator, map visualization.
7. Request Tracking & Lifecycle Management
- **Working:** Users can track request status (Pending, Verified, In Progress, Completed) in real-time. Updates are synchronized instantly via Firestore.
 - **Key UI Components:** Status tracker panel, progress indicator.
8. Notifications (Future Enhancement)
- **Working:** Sends emergency alerts, donor confirmations, and completion updates using Firebase Cloud Messaging to ensure timely communication.

4] USER INTERFACE (UI) COMPONENTS:

1. **Sign-Up Page** – Input fields for name, email, mobile number, role selection, and verification confirmation.

2. **Login Page** – Email & password input, password reset link.
3. **Receiver Dashboard** – Request creation button, request history, status tracker, notifications panel.
4. **Donor Dashboard** – Availability toggle, matched requests list, notification panel.
5. **NGO/Admin Dashboard** – Pending request list, verification controls, system monitoring panel.
6. **Emergency Request Page** – Form for creating emergency requests with urgency and location details.
7. **Map View Page** – Displays nearby donors and emergency locations using geolocation integration.

4.4: NAMING CONVENTIONS:

1) Sign-Up and Login Pages:

- **Function: handleRoleSelection ()**

Purpose: Allows users to select their role (Donor, Receiver, NGO, Admin) during registration.

Reason for Creation: Ensures proper role-based access control and dashboard routing after authentication.

- **Function: handleSignUpSubmit ()**

Purpose: Handles user registration by collecting validated input data and storing it in Firebase Authentication and Firestore.

Reason for Creation: Essential for securely creating new accounts and maintaining structured user data for different roles.

- **Function: handleLoginSubmit ()**

Purpose: Authenticates user credentials via Firebase Authentication and grants access to the appropriate role-based dashboard.

Reason for Creation: Required for secure login and controlled access to emergency request and donor management features.

2) Dashboard Module:

- **Function: fetchUserProfile ()**

Purpose: Retrieves user profile details (name, role, location, availability status, etc.) from Firestore.

Reason for Creation: Ensures that the dashboard displays accurate and up-to-date user information.

- **Function: fetchUserRequests ()**

Purpose: Retrieves emergency requests created by the receiver or assigned to a donor.

Reason for Creation: Enables users to monitor request history and status updates in real-time.

- **Function: navigateToModule(moduleName)**

Purpose: Redirects users to specific modules such as Create Request, Donor Dashboard, Verification Panel, or Notifications.

Reason for Creation: Improves navigation flow and enhances overall user experience.

3) Emergency Request Module:

- **Function: createEmergencyRequest ()**

Purpose: Saves a new emergency request (resource type, urgency, location, description) into Firestore with “Pending” status.

Reason for Creation: Core functionality allowing receivers to request urgent medical resources.

- **Function: `updateRequestStatus ()`**

Purpose: Updates the status of a request (Pending, Verified, In Progress, Completed).

Reason for Creation: Enables lifecycle tracking and ensures transparency throughout the process.

- **Function: `validateRequestData ()`**

Purpose: Ensures all required fields (resource type, urgency, location) are filled before submission.

Reason for Creation: Prevents incomplete or invalid emergency requests from entering the system.

4) Verification Module (NGO/Admin):

- **Function: `verifyRequest ()`**

Purpose: Reviews and validates emergency request authenticity before approval.

Reason for Creation: Prevents fraudulent or fake emergency posts and ensures system reliability.

- **Function: `approveUserVerification ()`**

Purpose: Verifies NGO or donor accounts requiring additional validation.

Reason for Creation: Maintains trust and authenticity within the platform.

5) AI Matching Module:

- **Function: `predictUrgency ()`**

Purpose: Uses AI logic to classify request urgency (Low, Medium, Critical).

Reason for Creation: Helps prioritize critical cases for faster donor notification.

- **Function: `matchNearestDonors ()`**

Purpose: Identifies and ranks nearby donors based on geolocation, availability, and urgency.

Reason for Creation: Reduces response time and improves matching efficiency.

- **Function: `calculateDistance ()`**

Purpose: Computes geographic distance between donor and receiver using latitude and longitude.

Reason for Creation: Supports accurate proximity-based donor selection.

6) Donor Module:

- **Function: `updateAvailabilityStatus ()`**

Purpose: Allows donors to toggle availability for emergency response.

Reason for Creation: Ensures only active donors receive notifications.

- **Function: `respondToRequest ()`**

Purpose: Allows donors to accept or reject an emergency request.

Reason for Creation: Enables real-time coordination between donor and receiver.

7) Notification Module:

- **Function: sendNotification ()**

Purpose: Sends real-time alerts for new emergency requests, verification approvals, and request status updates using Firebase Cloud Messaging.

Reason for Creation: Ensures immediate communication during emergencies.

- **Function: fetchUnreadNotifications ()**

Purpose: Retrieves unread notifications for display in the dashboard.

Reason for Creation: Keeps users updated without requiring manual refresh.

4.5 VALIDATIONS

1. Registration Validation: validateEmail ()

- Purpose: Ensures valid email format
- Criteria: username@domain.com

2. Mobile Validation: validateMobile ()

- Purpose: Ensures correct contact info
- Criteria: Exactly 10 digits

3. Role Validation: validateUserRole ()

- Purpose: Ensures correct access control
- Criteria: Must be Donor / Receiver / NGO / Admin

4. Request Validation: validateRequestData ()

- Purpose: Prevents incomplete emergency requests
- Criteria: Resource type, urgency, location mandatory

5. Urgency Validation: validateUrgencyLevel ()

- Purpose: Maintains correct prioritization
- Criteria: Low / Medium / Critical only

6. Donor Availability Validation

- Purpose: Ensures donors are active
- Criteria: Availability = true

7. Location Validation

- Purpose: Accurate donor matching
- Criteria: Valid city or GPS coordinates

8. Notification Validation

- Purpose: Prevents duplicate alerts
- Criteria: One active alert per donor per request

5. PROGRAM LISTING:

5.1 COST ESTIMATION — MEDI-REACH (Role-based MEDI-REACH Platform):

A) Domain Characteristics (Measurement Parameters)

MEASUREMENT PARAMETER	COUNT (value ≥ 0)	WEIGHTING FACTOR
Number of User Inputs (EI)	34	Average
Number of User Outputs (EO)	24	Average
Number of User Inquiries (EQ)	22	Average
Number of Files (ILF)	14	Complex
Number of External Interfaces (EIF)	6	Average

B) Complexity Adjustment Table (General System Characteristics)

ITEM	COMPLEXITY ADJUSTMENT QUESTIONS	No Influence (0)	1	2	3	4	Essential (5)
1	Does the system require reliable backup and recovery?	☐	☐	☐	☒	☐	☐
2	Are data communications required?	☐	☐	☒	☐	☐	☐
3	Are there distributed processing functions?	☐	☐	☒	☐	☐	☐
4	Is performance critical?	☐	☐	☐	☒	☐	☐
5	Will the system run in an existing, heavily utilized operational environment?	☐	☐	☒	☐	☐	☐
6	Does the system require on-line data entry?	☐	☐	☐	☒	☐	☐
7	Does the on-line data entry require the input transaction to be built over multiple screens or operations?	☐	☐	☐	☒	☐	☐
8	Are the master files updated	☐	☐	☐	☒	☐	☐

ITEM	COMPLEXITY ADJUSTMENT QUESTIONS	No Influence (0)	1	2	3	4	Essential (5)
	on-line?						
9	Are the inputs, outputs, files or inquiries complex?	☐	☐	☐	☒	☐	☐
10	Is the internal processing complex?	☐	☐	☐	☒	☐	☐
11	Is the code designed reusable?	☐	☐	☐	☒	☐	☐
12	Are conversion and installation included in the design?	☐	☒	☐	☐	☐	☐
13	Is the system designed for multiple installations in different organizations?	☐	☐	☒	☐	☐	☐
14	Is the application designed to facilitate change and ease of use by the user?	☐	☐	☐	☒	☐	☐

FP CALCULATION

RESULT	VALUE
Unadjusted Function Points (UFP)	305
Value Adjustment Factor (VAF)	1.04
PROJECT FUNCTION POINTS (AFP)	317.20

5.2 SCHEMA DESIGN:

1. Users collection

document id: userid (authentication uid)

- Name: full name of the user
- Age: age of the user
- Gender: gender of the user
- Email: user's email address
- Phone: contact number
- Role: donor | receiver | ngo | admin
- Bloodgroup (for donors)
- OrganizationName (for NGOs)
- Location: city / area of residence
- Availability: true / false (for donors)
- Verified: verification status (true/false)
- Dateofjoining: date of registration

→ subcollections:

1. EmergencyRequests (for receivers)

document id: requestid (unique id)

- ResourceType: blood | oxygen | plasma | medicine
- Urgency: low | medium | critical
- Location: emergency location
- Description: additional details about request
- Status: pending | verified | inprogress | completed | rejected
- CreatedAt: timestamp of request
- MatchedDonors: list of donor ids

2. DonorResponses (for donors)

document id: responseid

- Requestid: related emergency request
- Donorid: id of responding donor
- ResponseStatus: accepted | rejected
- ResponseTime: timestamp of response

3. VerificationLogs (for NGO/Admin)

document id: verificationid

- Requestid: id of emergency request
- Verifiedby: ngo/admin id

- VerificationStatus: approved | rejected
- Remarks: comments or notes
- VerifiedAt: timestamp

2. Admin collection

document id: adminid (unique id)

fields:

- Name: admin's full name
- Email: admin's email
- Gender: gender of the admin
- Staffid: unique staff id
- Role: system admin | verification manager
- Issuperadmin: boolean (true/false)

3. Notifications collection

document id: userid (unique per user)

→ subcollections:

1. EmergencyAlerts

document id: alertid

fields:

- Requestid: related emergency request
- ResourceType: type of resource requested
- Urgency: emergency level
- Location: request location
- Alertdate: date notification was sent
- ReadStatus: true/false

2. RequestUpdates

document id: updateid

fields:

- Requestid: related request
- Status: pending | verified | inprogress | completed
- Updatedate: date of status change

3. VerificationUpdates

document id: verificationid

fields:

- Requestid: related request
- VerificationStatus: approved | rejected
- Remarks: verification notes
- Date: date of update

4. GeneralAnnouncements

document id: announcementid

fields:

- AnnouncementText: general announcement / awareness message
- AnnouncementDate: date provided

4. Registry collections (for verification & authenticity)

1. DonorRegistry (for donors)

document id: donorid

- Name
- Bloodgroup
- Location
- Phone
- Email
- VerificationStatus

2. NGOREgistry (for NGOs)

document id: ngoid

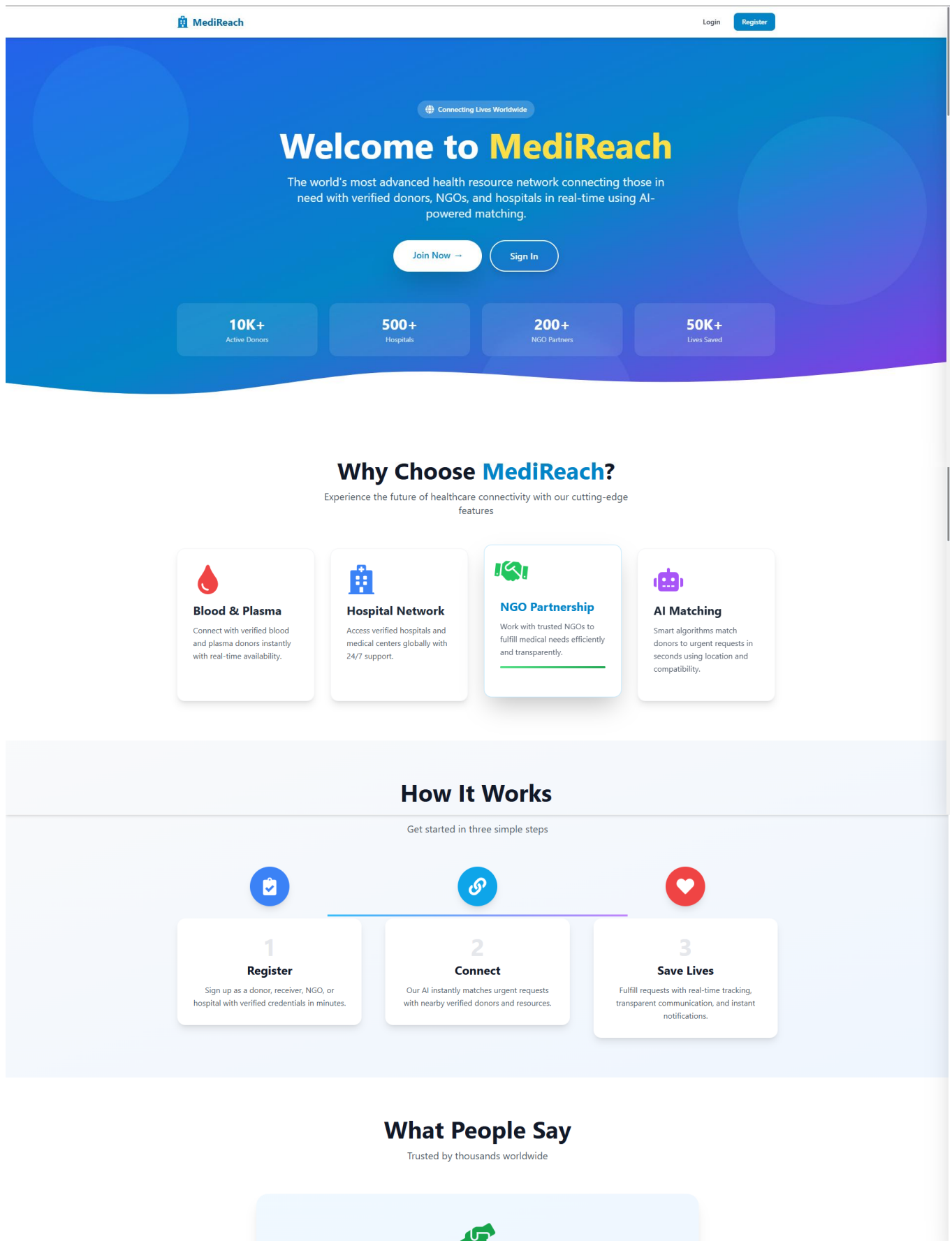
- Name
- OrganizationName
- RegistrationNumber
- Email
- Phone
- Location

3. EmergencyResourceRegistry (optional future integration)

document id: resourceid

- ResourceType
- AvailabilityCount
- Location
- OrganizationName

5.3 User Manual with Screenshots:



Ready to Make a Difference?

Join thousands of donors and organizations saving lives every day. Your contribution matters!

[Get Started Today](#)[Explore Donors](#)

MediReach

Connecting lives through technology, compassion, and instant healthcare resource matching worldwide.



Quick Links

- About Us
- Find Donors
- Become a Donor
- Urgent Requests

Resources

- Blog
- FAQ
- Support Center
- Partner With Us

Contact Us

- 123 Health Street, Medical District, NY 10001
- info@medireach.com
- +1-800-MEDIREACH
- 24/7 Emergency Support

© 2025 MediReach. All rights reserved. Saving lives, one connection at a time.

[Privacy Policy](#) [Terms of Service](#) [Cookie Policy](#)



Join MediReach

Start saving lives today

Create Your Account

Join our global health network

I am a *

Donor (Blood, Plasma, Resources)

Full Name *

John Doe

Email Address *

john@example.com

Password *

Min. 6 characters

Confirm Password *

Re-enter password

Phone Number *

+1 234 567 8900

Location *

City, Country

Donor Information

Blood Group *

Select Blood Group

☐ I am registered on NOTTO (National Organ & Tissue Transplant Organisation)

Check this if you're an official organ/tissue donor registered with NOTTO

Not registered yet? Become an official organ donor and save lives!

[Register on NOTTO](#)

Create Account

Already have an account? [Sign In](#)

[Back to Home](#)



MediReach

Global Health Resource Network

Welcome Back!

Sign in to continue saving lives

Email Address

your.email@example.com

Password

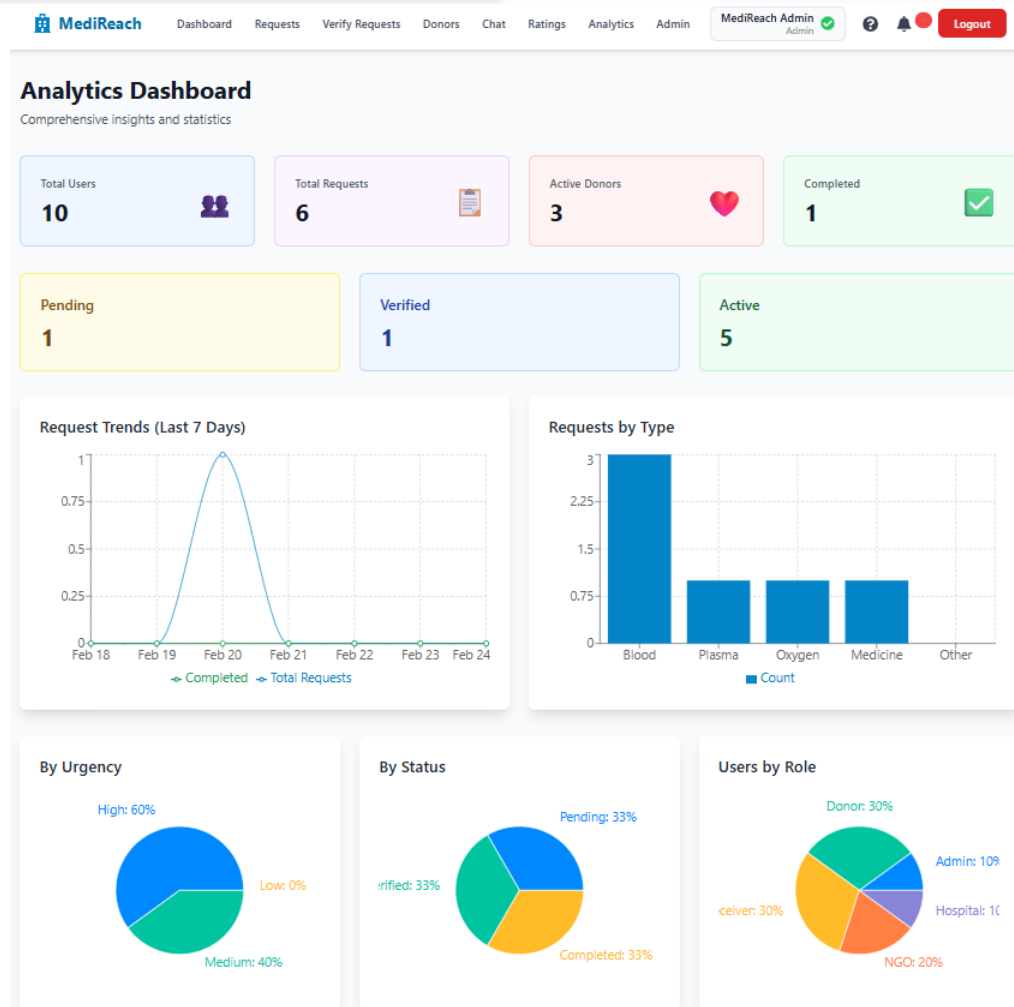
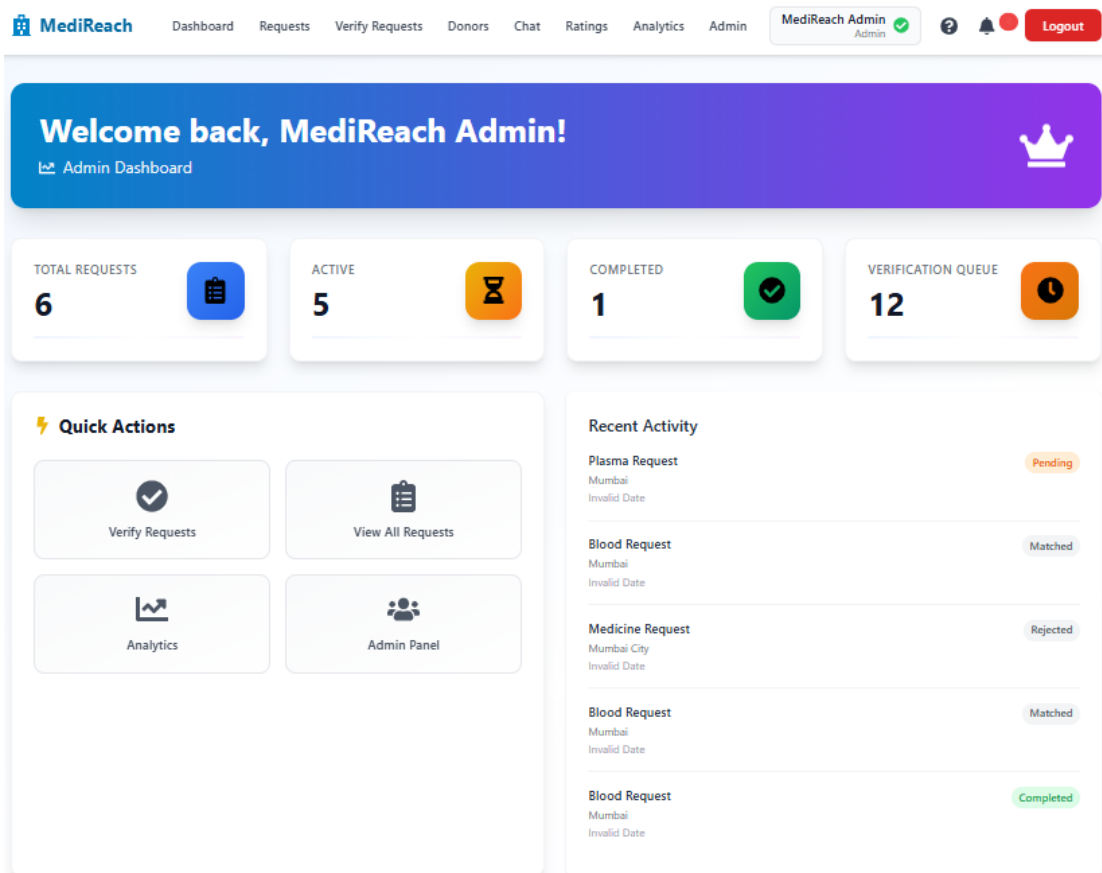
Enter your password

Sign In

Or continue with

[Continue with Google](#)

Don't have an account? [Create Account](#)



SOS

Create Emergency Request

Help is on the way - submit your medical resource request

Resource Type *

Urgency Level *

Blood

High

Location *

Mumbai

Description *

Please provide details about your medical emergency, quantity needed, blood group (if applicable), patient condition, etc.

Please provide as much detail as possible to help donors and verifiers understand your situation.

Note: Your request will be reviewed by our admin/NGO team for verification before being visible to donors. You'll receive a notification once verified.

Submit Emergency Request

Back to Requests

Donor Directory

Find verified donors near you

Search

Name, location...

Blood Group

All Blood Groups

Availability

All Donors

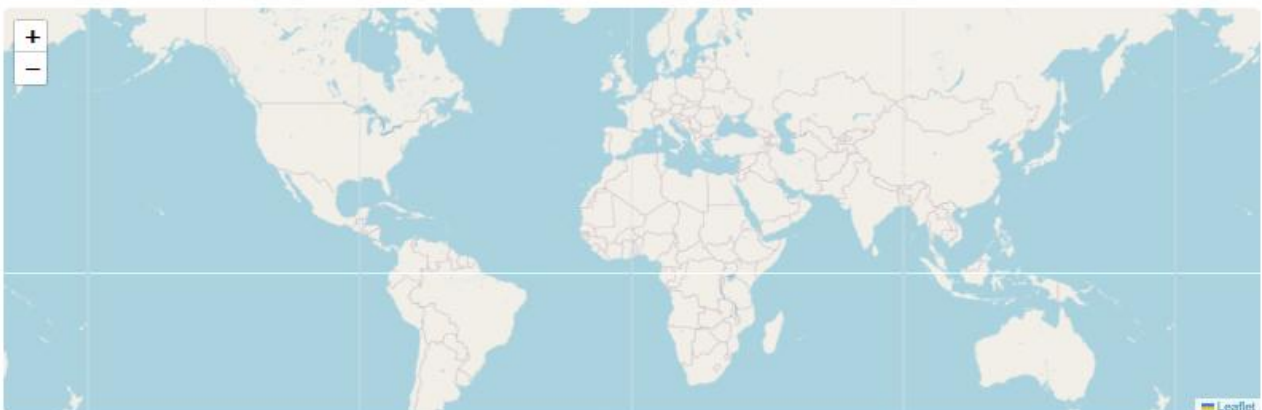
View



List

Map

Showing 3 of 3 donors



5.4 Test Case design:

Test Code	TEST DESCRIPTION	TEST INPUT	EXPECTED RESULT	ACTUAL RESULT
TC01	User Registration	Valid donor/receiver/NGO/admin details with correct role selection	User account created successfully and stored in database	As expected,
TC02	Invalid Registration	Missing required fields or invalid email format	Registration denied with validation error message	As expected,
TC03	User Login	Valid email & password	User successfully logged in and redirected to respective role-based dashboard	As expected,
TC04	Invalid Login	Incorrect email or password	Login denied with appropriate error message	As expected,
TC05	Create Emergency Request	Receiver submits valid resource type, urgency, and location	Request stored in database with status "Pending"	As expected,
TC06	Request Verification	NGO/Admin approves pending request	Request status updated to "Verified"	As expected,
TC07	AI Donor Matching	Verified emergency request	Nearest available donors identified and notified	As expected,
TC08	Donor Accepts Request	Donor clicks "Accept"	Request status updated to "In Progress" and receiver notified	As expected,
TC09	Request Completion	Receiver confirms resource received	Request status updated to "Completed"	As expected,
TC10	Donor Availability Update	Donor toggles availability status	Availability updated and reflected in matching system	As expected,
TC11	Notification Delivery	Verified emergency request	Real-time notification sent to matched donors	As expected,
TC12	Unauthorized Access Attempt	User tries to access another role's dashboard	Access denied with proper authorization error	As expected,

6. CONCLUSION:

6.1 CONCLUSION:

Medi-Reach is a web-based emergency medical resource and donor network designed to connect people in urgent medical need with verified donors, NGOs, hospitals, and administrators on a single digital platform. The system ensures secure authentication, role-based access, and verified request handling to reduce misinformation and delays during medical emergencies. Receivers can raise emergency requests, donors can respond based on availability, and NGOs or administrators can verify and monitor requests to ensure authenticity. By leveraging modern web technologies such as Firebase authentication, real-time databases, geolocation services, and AI-based donor matching, Medi-Reach improves emergency response efficiency and transparency. The platform minimizes dependency on unverified social media posts, reduces response time, and enhances coordination between all stakeholders. Medi-Reach aims to provide a reliable, scalable, and technology-driven solution for emergency medical assistance and humanitarian support.

6.2 LIMITATIONS:

- **Dependency on User Availability:** The effectiveness of Medi-Reach depends on the availability and willingness of donors. Lack of nearby donors may delay assistance in critical cases.
- **Internet Dependency:** The platform requires a stable internet connection for real-time data synchronization, notifications, and map-based services. Limited connectivity may reduce effectiveness in remote areas.
- **Verification Time:** During peak emergencies, verification by NGOs or admins may take time, potentially delaying request activation.
- **Location Accuracy:** Donor matching relies on location data, which may not always be accurate due to GPS limitations or incorrect user inputs.
- **Data Security Risks:** Since the system handles sensitive personal and medical information, strict security practices are required. Any misconfiguration could pose privacy risks.
- **Scalability Challenges:** As user volume increases, performance optimization and infrastructure scaling may be required to maintain system responsiveness.

6.3 FUTURE SCOPE OF THE PROJECT:

- **Government Integration:** Integration with national health platforms and blood bank registries to enable verified, nationwide donor access.
- **Advanced AI Models:** Implement more advanced machine learning models for urgency prediction, donor reliability scoring, and response optimization.
- **Mobile Application:** Develop native Android and iOS applications to improve accessibility and real-time responsiveness.
- **Disaster Management Integration:** Extend Medi-Reach to support large-scale disaster relief operations and emergency response coordination.
- **Multilingual Support:** Add support for multiple regional and international languages to improve accessibility.
- **Blockchain-Based Verification:** Implement blockchain technology for immutable request verification and donation tracking.
- **Wearable & IoT Integration:** Integrate health sensors and wearable devices for real-time emergency detection and alerts.

7. **BIBLIOGRAPHY:**

- World Health Organization (WHO) – Emergency Healthcare & Blood Donation Guidelines- <https://www.who.int/>
- Firebase Documentation – Google Developers - <https://firebase.google.com/docs>
- Google Maps API Documentation – Google Cloud - <https://developers.google.com/maps/documentation>
- Node.js Official Documentation - <https://nodejs.org/docs/latest/api/>
- React.js Official Documentation - <https://react.dev/>
- Research Papers on Emergency Medical Response Systems
- Articles on AI-Based Donor Matching and Healthcare Platforms
- NGO and Disaster Management Best Practice Reports